SeniorPadi

Learn it, run it, change it

Senior Paddi: Beginner's Project Guide

A friendly, step-by-step tour of the whole SeniorPadi website: what every folder and file does, how EJS pages are built, and how to run and change the project on your own.

Who this is for

Anyone who is new to Node.js, Express and EJS. No experience needed, only curiosity.

VERSION	DATE	PROJECT
1.0	19 October 2026	seniorpadi 0.1.0

Contents

Chapter 1Welcome, and how to use this guide 5

What SeniorPadi is 5

Who this guide is for 5

How to read it 6

How the boxes work 6

Chapter 2The big picture 8

A website is a conversation 8

The main parts of SeniorPadi 9

One visit, from start to finish 10

What happens when the server starts 11

Chapter 3Technologies used, in plain English 13

JavaScript and Node.js 14

npm and package.json 14

Express 15

EJS (Embedded JavaScript templates) 15

PostgreSQL and the pg driver 16

Sequelize, models and migrations 16

Sessions: express-session and connect-pg-simple 17

bcrypt (password hashing) 17

csrf-sync (form protection) 18

express-rate-limit 18

dotenv and environment variables 18

Uploads: multer and Cloudinary 19

Brevo (SMS and email) and Google Translate 19

node-cron (scheduled jobs) 19

qrcode 19

Socket.IO (live chat) 20

Tailwind CSS 20

Font Awesome and the Outfit font 20

Jest and supertest (tests) 21

Git, GitHub, Render and Neon 21

Chapter 4Project setup, step by step 22

1. Install the tools 22

2. Get the code 22

3. Install the packages 23

4. Create the database 23

5. Create your settings file (.env) 23

6. Check the database connection 25

7. Start the app 25

8. Add demo data (optional) 26

9. Open it in the browser 26

10. Stop and restart 26

Looking at pages without a database 27

Common setup errors 27

Chapter 5The folder and file map 29

The rule of thumb 29

Every file, grouped 30

Chapter 6EJS explained for beginners 41

What EJS is, and why it exists 41

The four EJS tags used in this project 41

How the server sends data to a view 43

Following one real page: Log in 44

Partials: reusing pieces 45

Loops and if/else: the event cards 46

Common beginner mistakes 48

Chapter 7File-by-file walkthrough 49

Starting up 49

Middleware 51

Routes 53

Services (the rules) 55

Models (the database tables) 57

Views (the pages) 59

Browser scripts, styles, words and tests 62

Chapter 8How the main features work, end to end 64

Creating an account 64

Logging in and out 64

The home page, hero and Events near you 65

Finding, filtering and joining events 66

CHAPTER 1

Welcome, and how to use this guide

This guide explains the SeniorPadi website from the ground up. By the end you will know how to run it on your own computer, find your way around its folders, and make changes with confidence.

What SeniorPadi is

SeniorPadi is a free website for people aged 60 and over in Ibadan, Nigeria. It helps them find community events near them (exercise groups, church programmes, skills classes, health talks and social gatherings), sign up to attend or volunteer, and, once staff have checked their ID, meet and chat with other seniors.

The problem it solves is **loneliness and not knowing what is on nearby**. Many of the people using it have simple phones, slow internet and sometimes a relative helping them. So the site is built to be calm, readable and forgiving: big text, big buttons, plain words, and it works in English and Yorùbá.

There are two kinds of people who use it:

- **Seniors** sign up with a phone number, browse events, register, get a confirmation text and a reminder, and can message other verified seniors.
- **Staff** (people at the NGO that runs the site) add organizations and events, mark who came, check IDs and handle reports. In the code, staff are called `admin`.

> **IMPORTANT**
>
> Everything in this guide was checked against the real project files on the `main` branch. Every code snippet you see is copied straight out of the project by the program that builds this PDF, with its file name and line numbers. If a snippet ever stops matching, the build refuses to finish, so the guide cannot quietly go out of date.

Who this guide is for

You, if you are smart and curious but new to some or all of these: JavaScript on a server, Node.js, Express, EJS templates, databases and Tailwind CSS. You do not need any of them before you start. Every technical word is explained the first time it appears, and there is a glossary at the back (chapter 12).

How to read it

If you want to...	Read
Understand the whole thing	Chapters 1 to 9 in order. Each one builds on the last.
Just get it running today	Chapter 2 (the big picture) and chapter 4 (setup). Come back to the rest later.
Change some text, a colour or a picture	Chapter 10 (recipes). Each recipe names the exact files to open.
Fix an error message	Chapter 11 (troubleshooting).
Look something up quickly	Chapter 12 (glossary) and chapter 13 (cheat sheet).

You can safely skip chapter 7 (file by file) on a first read. It is a reference: open it when you need to know what a particular file does.

How the boxes work

> **TIP**
> A shortcut or a good habit.

> **WARNING**
> Something that can break the site or lose data if you are not careful.

> **IMPORTANT**
> A rule you need to remember.

> **TRY IT YOURSELF**
> A small hands-on task. Doing them is the fastest way to learn.

> **THINGS TO KNOW**
> Something in the project that is unclear, unfinished or looks like a bug. These are collected again at the end of chapter 11.

Code appears in boxes like this one, with the file it came from on the dark bar and plain-English notes underneath:

package.json, line 7–8

```
"start": "node src/server.js",
"dev": "nodemon src/server.js --ext js,json",
```

- `"start"` is a name you can run with `npm start`. It starts the website with Node.js.
- `"dev"` runs it with *nodemon*, a helper that restarts the site every time you save a file.

Words you type in a terminal look like this:

```
npm start
```

What you learned

- SeniorPadi helps people aged 60+ in Ibadan find events, register, and meet other seniors. Staff run it.
- The guide matches the real code on `main`; snippets are copied from the files automatically.
- Chapters 2 and 4 get you running; chapter 10 is for making changes.

CHAPTER 2

The big picture

Before looking at any code, it helps to see the whole machine at once. This chapter follows one visit, from typing an address to seeing a page.

A website is a conversation

Every time you open a page, two programs talk to each other:

- The **browser** (Chrome, Edge, the browser on a phone) is the program that shows pages. It *asks*.
- The **server** is a program running on a computer somewhere (or on your own computer while you work). It *answers*.

The question is called a **request** and the answer is called a **response**. A request is like a letter with an address on it, for example `GET /events` ("please give me the events page"). `GET` means "show me something"; `POST` means "here is a form I filled in".

> **AN EVERYDAY PICTURE**
>
> Think of SeniorPadi as a small office. The **address** on the letter (`/login`) is like a street address. The **route** is the person who answers the door at that address. The **database** is the filing cabinet in the back room. The **view** is a pre-printed form that the person fills in before handing it back to you.

The main parts of SeniorPadi

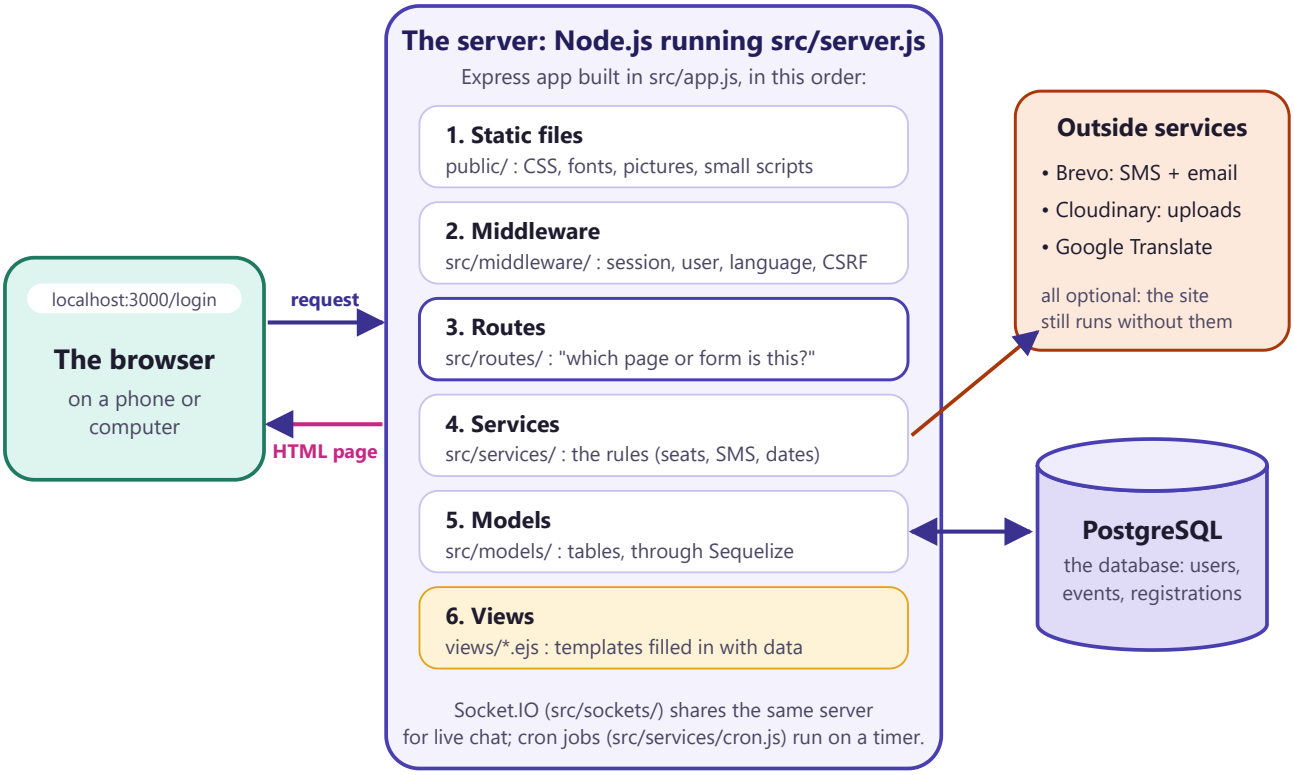


Figure 2.1 How the parts of SeniorPadi connect. The numbers are the order a request passes through.

Part	Where it lives	What it does, in one sentence
Server start	`src/server.js`	Checks the settings, connects to the database, then starts listening for requests.
The app	`src/app.js`	Builds the Express app and decides the order every request is handled in.
Static files	`public/`	Files sent to the browser exactly as they are: the stylesheet, fonts, pictures and small scripts.
Middleware	`src/middleware/`	Small checks that run before every page: who is logged in, which language, is the form genuine.
Routes	`src/routes/`	One function per address, which decides what to do and which page to show.
Services	`src/services/`	The project's rules, kept out of the routes so several routes can share them (seats, SMS, dates).
Models	`src/models/`	A JavaScript description of each database table, used to read and save data.

Part	Where it lives	What it does, in one sentence
Database	PostgreSQL (outside the project)	Where users, events and registrations are stored permanently.
Views	`views/`	EJS templates: HTML pages with gaps that are filled in with data.
Words	`locales/`	Every sentence on the site, in English and Yorùbá.

One visit, from start to finish

Here is what happens when someone types `localhost:3000/login`, fills in the form and presses **Log in**. Every page on the site follows this same path; only the route, the data and the view change.

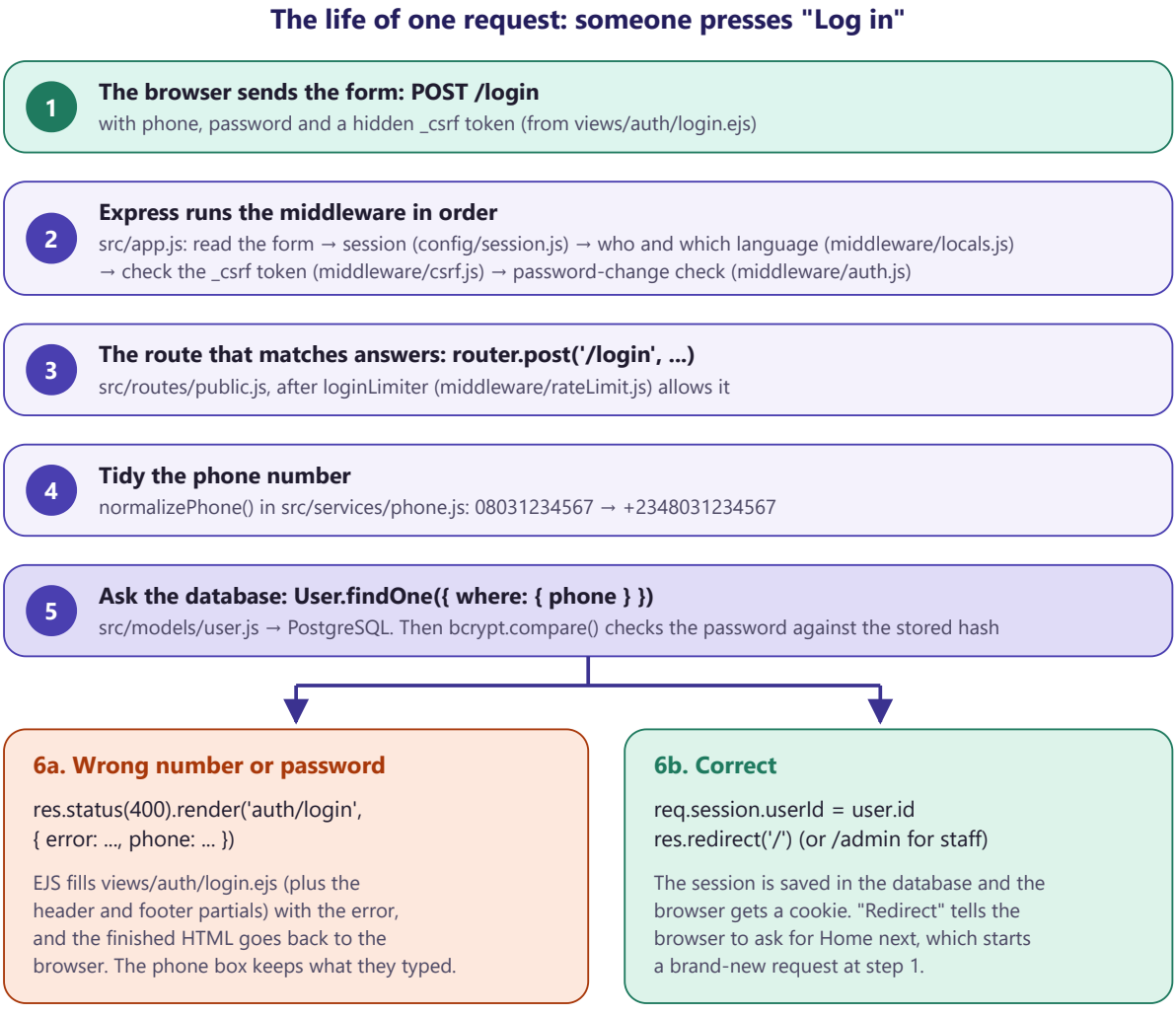


Figure 2.2 The life of one request. The file names show where each step happens.

Opening the page (GET /login)

1. The browser asks for `GET /login`.
2. `src/app.js` first checks whether `public/` has a file called `login`. It doesn't, so the request moves on.
3. The middleware runs: the session is loaded (`src/config/session.js`), the user and language are found (`src/middleware/locals.js`) and a fresh security token is made for the form (`src/middleware/csrf.js`).
4. The routes are tried in order until one matches. `src/routes/public.js` has `router.get('/login', ...)`, so it answers.
5. That route calls `res.render('auth/login', {...})`. EJS takes `views/auth/login.ejs`, fills in the words and the token, and produces plain HTML.
6. The HTML goes back to the browser. The browser then asks for the files the page mentions (`/css/main.css`, the font, the scripts) and those come straight from `public/`.

Sending the form (POST /login)

Pressing **Log in** sends a second request, `POST /login`, with what was typed. Figure 2.2 shows its steps. The important idea is the ending: if the details are wrong, the server shows the same page again with an error message; if they are right, it remembers the person in the **session** and sends a **redirect**, which tells the browser to go and ask for another page (Home).

> **IMPORTANT**
>
> The server builds the whole page before sending it. This is called **server-side rendering**. It is why SeniorPadi works even on old phones and even with JavaScript switched off: the browser only has to show finished HTML. The small scripts in `public/js/` just make things nicer (pop-ups, the password eye button); every form still works without them.

What happens when the server starts

When you run `npm start`, `src/server.js` does its set-up work in this order. You can watch it happen in the terminal.

`src/server.js`, line 8–19

```
async function start() {
  console.log('\nStarting SeniorPadi...');

  const warnings = startup.prepareEnvironment();

  // these read the settings, so they're loaded after prepareEnvironment
  const http = require('http');
  const models = require('./models');
```

```
  const { ensureDatabase } = require('./services/migrate');

  await startup.waitForDatabase(models.sequelize);
  startup.line('Database connected (' + models.sequelize.config.host + ')');
```

- `prepareEnvironment()` reads your `.env` settings and fills in safe defaults (see `src/services/startup.js`).
- `require('./models')` loads every table description and connects Sequelize to the database.
- `waitForDatabase` tries to connect up to six times, waiting a little longer each time.
- `ensureDatabase` creates or updates the tables (it runs the "migrations", explained in chapter 3).

After that it adds the welcome-guide text and the first staff account if they are missing, rebuilds the stylesheet if it is missing, plugs in live chat, starts the timed jobs and finally opens the door:

src/server.js, line 52–56

```
const port = process.env.PORT || 3000;
server.listen(port, () => {
  for (const warning of warnings) console.log('  ! ' + warning);
  console.log('SeniorPadi is running at ' + process.env.APP_URL + '\n');
});
```

- The site listens on the port number in your settings, or **3000** if there is none. That is why the address is `localhost:3000`.
- Any warnings (for example "Brevo is not set up") are printed first, so you see them.

TRY IT YOURSELF

Once you have the site running (chapter 4), open `http://localhost:3000/health`. This tiny route in `src/routes/public.js` asks the database "are you there?" and answers `{"ok":true}`. It is how Render and uptime checkers know the site is alive.

What you learned

- The browser sends a **request**; the server sends back a **response** (usually a finished HTML page).
- Inside the server a request passes through static files, middleware, routes, services, models and views, in that order.
- A **redirect** tells the browser to ask for a different page; a **session** is how the site remembers who you are.
- `src/server.js` sets everything up on its own each time it starts.

CHAPTER 3

Technologies used, in plain English

SeniorPadi is built from a handful of well-known tools. For each one: what it is, why the project uses it, where you will find it, and a tiny real example from the code.

The version numbers come from `package.json` and are the ones installed today. A version written as `^5.2.1` in `package.json` means "5.2.1 or any newer 5.x".

Technology	Version	Job in one line
JavaScript	(language)	The programming language for both the server and the small browser scripts.
Node.js	20 or newer	Runs JavaScript on a computer instead of in a browser.
npm	comes with Node	Downloads packages and runs the project's commands.
Express	5.2.1	The web framework: turns addresses into pages.
EJS	6.0.1	Templates: HTML with gaps for data.
PostgreSQL + pg	pg 8.23.0	The database, and the driver Node uses to talk to it.
Sequelize (+ CLI)	6.37.8 / 6.6.5	Lets you use database tables as JavaScript objects.
express-session + connect-pg-simple	1.19.0 / 10.0.0	Remembers who is logged in, stored in the database.
bcrypt	6.0.0	Scrambles passwords so they are never stored readable.
csrf-sync	4.2.1	Protects forms from being faked by other websites.
express-rate-limit	8.7.0	Slows down people guessing passwords.
dotenv	18.0.4	Reads private settings from the `.env` file.
multer	2.4.0	Receives uploaded files.
cloudinary	2.11.0	Stores pictures, ID documents and videos online.
node-cron	4.6.0	Runs jobs on a timetable (reminder texts at 8am).
qrcode	1.5.4	Draws the optional QR check-in code.
Socket.IO	4.8.4	Live chat: new messages appear without reloading.
Tailwind CSS (+ CLI)	4.3.3	The styling system that builds the stylesheet.

Technology	Version	Job in one line
Font Awesome	6.5.2 (from a CDN)	The little icons.
Brevo, Google Translate	web services	SMS and email; Yorùbá translations of event text.
Jest + supertest	30.5.2 / 7.3.0	Automatic tests.
nodemon, cross-env	3.1.14 / 10.1.0	Development helpers.
Git, GitHub, Render, Neon	tools and services	Saving history, sharing code, hosting the site and the database.

JavaScript and Node.js

JavaScript is a programming language. It started in web browsers, which is why the little scripts in `public/js/` are JavaScript. **Node.js** is a program that runs JavaScript on an ordinary computer or server, outside a browser. All of `src/` runs in Node.js. The project says it needs version 20 or newer:

`package.json`, line 62–64

```
"engines": {
  "node": ">=20"
}
```

Two JavaScript ideas you will see everywhere in this project:

- `require('...')` loads another file or package so you can use it, like borrowing a tool from a shelf. `module.exports = ...` is how a file puts its tools on the shelf.
- `async` and `await`: talking to the database takes time, so `await` means "wait here for the answer before carrying on".

npm and package.json

npm (Node Package Manager) comes with Node.js. A **package** is code someone else wrote and shared, like Express. `package.json` lists the packages SeniorPadi needs and gives names to its commands ("scripts"). `npm install` downloads the packages into `node_modules/`; `npm run dev` runs the script called `dev`.

`package.json`, line 9

```
"dev:local": "cross-env DB_TARGET=local nodemon src/server.js --ext js,json",
```

- This script uses **cross-env** to set `DB_TARGET=local` (works on Windows, Mac and Linux alike) and **nodemon** to restart the site when a `.js` or `.json` file changes.

Express

Express is a *web framework*: a ready-made skeleton for a web server. You tell it "when someone asks for this address, run this function". Those instructions are called **routes**. Express also runs **middleware**, functions that every request passes through on its way to a route, like security guards in a corridor.

Where: `src/app.js` builds the app; `src/routes/` holds the routes; `src/middleware/` holds the middleware.

src/app.js, line 29–35

```
app.use('/', require('./routes/public'));
app.use('/', require('./routes/verify'));
app.use('/', require('./routes/events'));
app.use('/', require('./routes/account'));
app.use('/', require('./routes/social'));
app.use('/', require('./routes/onboarding'));
app.use('/admin', require('./routes/admin'));
```

- Each line plugs a file of routes into the app. They are tried top to bottom until one matches the address.
- The admin routes are plugged in at `/admin`, so `router.get('/events')` inside `admin.js` answers `/admin/events`.

EJS (Embedded JavaScript templates)

EJS lets you write an ordinary HTML page with special tags where data should go. The server fills in the gaps and sends plain HTML. Chapter 6 is all about EJS.

Where: every file in `views/`. It is switched on in `src/app.js`:

src/app.js, line 14–15

```
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '../views'));
```

- "Use EJS for pages" and "the pages are in the `views` folder". So `res.render('home')` means `views/home.ejs`.

PostgreSQL and the pg driver

A **database** stores information permanently, in **tables** (like spreadsheets): a Users table, an Events table and so on. Each row is one thing (one user); each column is one detail (their phone number). **PostgreSQL** ("Postgres") is the database program SeniorPadi uses. It runs separately from the website: on your computer while you develop, or on **Neon** (a Postgres host in the cloud) for the live site. **pg** is the package Node uses to talk to it.

src/config/session.js, line 9

```
const sessionPool = new Pool({ connectionString: db.url, ssl: db.ssl || false, max: 3 });
```

- A "pool" is a small set of open connections to the database that get reused, which is faster than connecting for every request.

Sequelize, models and migrations

Sequelize is an *ORM* (object-relational mapper): it lets you work with tables using JavaScript instead of writing database commands (SQL) by hand. A **model** is the JavaScript description of one table.

src/models/event.js, line 6–10

```
const Event = sequelize.define('Event', {
  title: { type: DataTypes.STRING, allowNull: false },
  description: { type: DataTypes.TEXT, allowNull: false },
  imageUrl: DataTypes.STRING,
  imagePublicId: DataTypes.STRING,
```

Reading data then looks like this (the Home page asking for the next four published events):

src/routes/public.js, line 36–41

```
const upcoming = await Event.findAll({
  where: { status: 'published', startsAt: { [Op.gt]: new Date() } },
  include: [Organization],
  order: [['startsAt', 'ASC']],
  limit: 4
});
```

- `where`: only published events that start after now (`Op.gt` means "greater than").
- `include: [Organization]`: also fetch the organization each event belongs to.
- `order` and `limit`: soonest first, at most four.

A **migration** is a small file that changes the shape of the database, for example "create the Organizations table". They are kept in date order in `src/migrations/` and each one runs only once. SeniorPadi runs new migrations by itself every time it starts (`src/services/migrate.js`), so

you rarely run them by hand. The **Sequelize CLI** (command-line interface) can also run them: `npm run db:migrate`. `.sequelizerc` tells it where things are.

src/migrations/20260927000002-create-organizations.js, line 5–9

```
await queryInterface.createTable('Organizations', {
  id: { type: Sequelize.INTEGER, primaryKey: true, autoIncrement: true },
  name: { type: Sequelize.STRING, allowNull: false },
  contactName: Sequelize.STRING,
  contactPhone: Sequelize.STRING,
```

Sessions: express-session and connect-pg-simple

The web forgets you between requests. A **session** is how the site remembers you: after you log in, the server stores `userId` in a session and gives your browser a **cookie** (a small label the browser sends back with every request). **express-session** handles this, and **connect-pg-simple** stores sessions in the database so they survive a restart.

src/config/session.js, line 21–26

```
cookie: {
  httpOnly: true,
  sameSite: 'lax',
  secure: isProduction,
  maxAge: 30 * 24 * 60 * 60 * 1000 // 30 days, seniors shouldn't have to log in often
}
```

- `httpOnly`: page scripts cannot read the cookie (safer).
- `maxAge`: 30 days, so seniors don't have to log in often.

bcrypt (password hashing)

Passwords are never stored as typed. **bcrypt** turns a password into a *hash*, a scrambled string that cannot be turned back. At log-in, bcrypt checks whether the typed password produces the same hash.

src/routes/public.js, line 160

```
const passwordOk = user && await bcrypt.compare(req.body.password || '',
user.passwordHash);
```

csrf-sync (form protection)

CSRF (cross-site request forgery) is a trick where another website secretly submits a form to SeniorPadi using your logged-in browser. To stop it, every form carries a secret **token** that only the real page knows, and the server rejects forms without the right one.

views/auth/login.ejs, line 30

```
<class="tk-tag">input type="hidden" name="_csrf" value="<%= csrfToken %>">
```

> **WARNING**
> Every `<form method="post">` you add must include this hidden `_csrf` line, or submitting it shows "This form has expired".

express-rate-limit

Limits how many times someone can try something in a period of time, so nobody can guess passwords by trying thousands.

src/middleware/rateLimit.js, line 17–21

```
loginLimiter: limiter(10, 15),
signupLimiter: limiter(5, 60),
resetLimiter: limiter(5, 60),
uploadLimiter: limiter(20, 60),
messageLimiter: limiter(60, 10)
```

→ Log in: at most 10 tries every 15 minutes. Sign up: 5 an hour, and so on.

dotenv and environment variables

An **environment variable** is a setting given to a program from outside its code, such as the database address or a password. Keeping them outside the code means secrets never end up on GitHub. **dotenv** reads them from a file called `.env` in the project folder; the code then reads them as `process.env.NAME`.

src/app.js, line 1

```
require('dotenv').config({ quiet: true });
```

Uploads: multer and Cloudinary

multer receives files sent from a form (profile pictures, ID documents, the welcome video) and keeps them in memory. **Cloudinary** is an online picture and video store; `src/services/cloudinary.js` sends the files there. If Cloudinary is not set up, that same file saves uploads on your computer instead (in `public/uploads/` and `storage/ids/`, both ignored by Git).

src/middleware/upload.js, line 11–12

```
storage: multer.memoryStorage(),
limits: { fileSize: maxMb * 1024 * 1024 },
```

Brevo (SMS and email) and Google Translate

These are outside web services the server calls over the internet with `fetch` (no extra package). **Brevo** sends the confirmation and reminder texts and emails; **Google Cloud Translation** translates event titles and descriptions into Yorùbá. Both are optional: without keys, messages are printed in the terminal instead, and event text stays in English.

src/services/notify.js, line 8–9

```
const BREVO_EMAIL_URL = 'https://api.brevo.com/v3/smtp/email';
const BREVO_SMS_URL = 'https://api.brevo.com/v3/transactionalSMS/sms';
```

node-cron (scheduled jobs)

Runs a function on a timetable, written in "cron" format. SeniorPadi sends reminders every morning at 8:00 and closes finished events every hour, in Lagos time.

src/services/cron.js, line 73–74

```
cron.schedule('0 8 * * *', runReminders, { timezone: TIME_ZONE });
cron.schedule('0 * * * *', runCompleteEvents, { timezone: TIME_ZONE });
```

→ `'0 8 * * *'` reads as "minute 0, hour 8, every day". `'0 * * * *'` is "minute 0 of every hour".

qrcode

Draws a QR code picture as SVG. Each registration has a QR code staff can scan to mark someone as arrived; it is always optional.

src/services/qr.js, line 10–12

```
function qrSvg(qrToken) {
  return QRCode.toString(scanUrl(qrToken), { type: 'svg', margin: 1, errorCorrectionLevel:
```

```
'M' });
}
```

Socket.IO (live chat)

Normally the browser has to ask before the server can answer. **Socket.IO** keeps a line open between them, so the server can push a new chat message the moment it arrives. It shares the same login session as the website. The plain message form still works if it fails.

src/sockets/index.js, line 31

```
io.engine.use(sessionMiddleware);
```

Tailwind CSS

CSS (Cascading Style Sheets) is the language for how a page looks: colours, sizes, spacing. **Tailwind CSS** is a CSS tool built around small ready-made class names such as `text-lg` (large text) or `rounded-full` (fully rounded corners), which you write straight into the HTML. You edit `src/styles/input.css`; the Tailwind **CLI** reads it plus every template, and writes the finished `public/css/main.css`.

src/styles/input.css, line 1–3

```
@import "tailwindcss";
@source "../../views";
@source "../../public/js";
```

- `@source` tells Tailwind which files to scan for class names (the views and the browser scripts).
- Brand colours and the font are set in an `@theme` block further down. Chapter 9 shows it.

Font Awesome and the Outfit font

The small icons (phone, calendar, lock) come from **Font Awesome**, loaded from a public *CDN* (a fast file server on the internet), so icons need an internet connection. The text font, **Outfit**, is stored in the project itself (`public/fonts/`).

views/partials/header.ejs, line 23

```
<class="tk-tag">link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.2/css/all.min.css" crossorigin="anonymous" referrerpolicy="no-referrer">
```

Jest and supertest (tests)

A **test** is a small program that uses the site and checks it behaves correctly. **Jest** runs the tests in `tests/`; **supertest** lets a test send requests to the app without a browser. Run them with `npm test`. They need their own empty database (`TEST_DATABASE_URL`) because they wipe it.

tests/routes.test.js, line 20–30

```
test('a senior can sign up with a plain form (no JavaScript)', async () => {
  const agent = request.agent(app);
  const page = await agent.get('/signup');
  const res = await agent.post('/signup').type('form').send({
    _csrf: csrfFrom(page.text),
    fullName: 'Mama Tolu',
    phone: '0805 123 4567',
    password: 'secret123',
    ageDeclared: 'yes'
  });
  expect(res.status).toBe(302);
```

Git, GitHub, Render and Neon

- **Git** keeps the history of every change, so you can always go back. A *commit* is a saved snapshot; a *branch* is a separate line of work.
- **GitHub** stores the Git history online and is where pull requests are reviewed and merged.
- **Render** runs the live website. `render.yaml` tells it how.
- **Neon** runs the live PostgreSQL database.

What you learned

- The server is JavaScript running in Node.js, organised by Express, with pages written in EJS.
- Data lives in PostgreSQL; Sequelize models describe tables, and migrations change them.
- Sessions, bcrypt, CSRF tokens and rate limits keep accounts safe.
- Tailwind builds `public/css/main.css` from `src/styles/input.css`.
- Brevo, Cloudinary and Google Translate are optional add-ons; the site runs without them.

CHAPTER 4

Project setup, step by step

This chapter takes you from an empty computer to SeniorPadi running in your browser. Follow the steps in order. Where Windows, macOS and Linux differ, each is shown.

> **TIP: WHAT A TERMINAL IS**
>
> A **terminal** is a window where you type commands instead of clicking. On Windows use **PowerShell** or the terminal inside VS Code (Ctrl + `). On macOS use **Terminal**. On Linux use your terminal app. Type a command, then press Enter.

1. Install the tools

Tool	Why	How
Node.js 20+ (LTS)	Runs the server. npm comes with it.	Download the LTS version from nodejs.org and install it. Check with `node -v` (should say v20 or higher).
Git	Downloads the code and keeps its history.	Windows: git-scm.com. macOS: run `git --version` and accept the offer to install. Linux: `sudo apt install git`.
PostgreSQL	The database.	Windows: the installer from postgresql.org (remember the password you choose for the `postgres` user). macOS: Postgres.app, or `brew install postgresql`. Linux: `sudo apt install postgresql`. Or skip it and use a free Neon database online.
A code editor	To read and edit files.	Visual Studio Code (code.visualstudio.com) is a good free choice.

2. Get the code

In a terminal, go to the folder where you keep projects and **clone** (download) the repository:

```
git clone https://github.com/Ask4mike/seniorpaddi-frontend.git
cd seniorpaddi-frontend
```

> **THINGS TO KNOW**
>
> This is the repository the project's Git is connected to (its `origin`). `README.md` mentions a different address (`Otani-ibe/SeniorPadi_Capstone`), which looks like an earlier home of the project. Use whichever one you were given access to.

3. Install the packages

```
npm install
```

This reads `package.json` and downloads everything into a new `node_modules/` folder. It can take a few minutes the first time. You only need to run it again when `package.json` changes (for example after pulling someone else's work).

4. Create the database

Skip this if you will use Neon. Otherwise make an empty database called `seniorpadi`. The tables are created for you later.

```
# macOS, Linux, or Windows when psql is on your PATH:
psql -U postgres -c "CREATE DATABASE seniorpadi;"
# Linux, if psql says "role postgres does not exist" or asks for a password you never set:
sudo -u postgres psql -c "CREATE DATABASE seniorpadi;"
```

On Windows, if `psql` is "not recognized", use its full path, for example `& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -c "CREATE DATABASE seniorpadi;"` (change 17 to your version), or create the database by right-clicking **Databases** in pgAdmin, which is installed alongside PostgreSQL.

5. Create your settings file (.env)

Create a new file called exactly `.env` (with the dot, no other extension) in the main project folder, next to `package.json`. Paste this in and replace the placeholder values:

```
# which database: local (PostgreSQL on this computer) or neon
DB_TARGET=local
LOCAL_DATABASE_URL=postgresql://postgres:YOUR_DB_PASSWORD_HERE@localhost:5432/seniorpadi
# NEON_DATABASE_URL=postgres://YOUR_NEON_USER:YOUR_NEON_PASSWORD@YOUR_NEON_HOST/neondb

SESSION_SECRET=YOUR_LONG_RANDOM_TEXT_HERE

# the first staff (admin) account, created on first start
ADMIN_NAME=Your Name
ADMIN_PHONE=08000000000
```

```
ADMIN_PASSWORD=YOUR_ADMIN_PASSWORD_HERE

# print texts and emails in the terminal instead of sending them
NOTIFY_DRY_RUN=true
SUPPORT_PHONE=08000000000
```

WARNING

- **Never commit** `.env` or paste it anywhere public. It holds passwords. It is already listed in `.gitignore`, so Git ignores it.
- If your database password contains `@`, `:` or `/`, it must be "URL-encoded" in the address (for example `@` becomes `%40`).

Every setting, explained

These are all the environment variables the code reads (found by searching `src/` for `process.env`). Only the database address is truly required.

Setting	What it is for
`DB_TARGET`	`local` or `neon`. Leave it out on Render, where `DATABASE_URL` is used.
`LOCAL_DATABASE_URL`	Address of PostgreSQL on your computer.
`NEON_DATABASE_URL`	Address of a Neon database.
`DATABASE_URL`	Address used when no `DB_TARGET` is set (the live site on Render).
`TEST_DATABASE_URL`	A *separate* database for `npm test`. It gets wiped every run.
`DB_SSL`, `DB_SSL_STRICT`	Only for unusual database hosts that need encrypted connections. Neon is handled automatically.
`SESSION_SECRET`	Long random text used to sign the login cookie. If missing, one is made up at each start (everyone is logged out on restart).
`ADMIN_NAME`, `ADMIN_PHONE`, `ADMIN_PASSWORD`	The first staff account, made automatically if there is no admin yet. Password must be 8+ characters.
`PORT`	Which port to listen on. Default `3000`.
`APP_URL`	The public address, used in links inside texts and emails. Default `http://localhost:3000`. Render fills it in through `RENDER_EXTERNAL_URL`.
`NODE_ENV`	`production` on the live site (secure cookies, caching). Leave it out on your computer.
`NOTIFY_DRY_RUN`	`true` = print texts and emails in the terminal instead of sending them.

Setting	What it is for
`BREVO_API_KEY`, `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`, `BREVO_SMS_SENDER`	Sending real SMS and email through Brevo.
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Storing uploads in Cloudinary.
`GOOGLE_TRANSLATE_API_KEY`	Translating event text into Yorùbá.
`SUPPORT_PHONE`, `SUPPORT_HOURS`	The help number and opening hours shown on the Call page.

6. Check the database connection

```
npm run db:check
```

This runs `src/scripts/checkDb.js`, which prints which database the app will use and whether it can reach it. You want to see `Connected: yes`. If not, chapter 11 lists the usual causes.

7. Start the app

Command	When to use it
`npm start`	Starts the site once. Uses whatever `DB_TARGET` says in `.env`.
`npm run dev`	Same, but restarts by itself when you save a `.js` or `.json` file. **Best while working.**
`npm run dev:local` / `dev:neon`	Like `dev`, but forces the local or Neon database whatever `.env` says.
Double-click `start-windows.bat`	Windows shortcut: checks Node, installs packages the first time, then runs `npm start`.

The first start creates every table and your admin account. A good first start looks like this:

```
Starting SeniorPadi...
  Database connected (localhost)
  Database updated: 10 change(s) applied
  Added the default welcome guide text
  Created the first admin account (08000000000)
  ! Brevo is not set up. Texts and emails are printed in the log instead of sent.
```

```
  ! Cloudinary is not set up. Picture, ID and video uploads are switched off.
  ! Google Translate is not set up. Menus are still in Yorùbá, but event text stays in
English.
SeniorPadi is running at http://localhost:3000
```

The lines with `!` are warnings, not errors: the site works without those services.

8. Add demo data (optional)

```
npm run seed:local
```

This **wipes your local database** and fills it with sample organizations, events and users (`src/scripts/seed.js`). It refuses to wipe a Neon database unless you use `npm run seed:neon` on purpose. The README lists the demo accounts: every one uses the password `password123`, and the demo admin's phone number is `08000000001`.

9. Open it in the browser

- The site: http://localhost:3000
- Log in: `http://localhost:3000/login`. Staff are sent to the dashboard at `/admin` after logging in.
- Health check: `http://localhost:3000/health`

`localhost` means "this computer". Only you can see it; nothing is on the internet.

10. Stop and restart

- **Stop:** click in the terminal that is running the site and press Ctrl + C (on macOS too). If asked "Terminate batch job (Y/N)?", type Y.
- **Start again:** run the same command (`npm run dev`). Pressing ↑ in the terminal brings back the last command.
- **After changes**, see the table below.

You changed...	What to do
A `.js` file in `src/`, or a `.json` file (like `locales/en.json`)	With `npm run dev`: nothing, it restarts itself. With `npm start`: stop and start.
An `.ejs` file in `views/`	Just refresh the browser. (On your computer, Express reads templates fresh every time.)

You changed...	What to do
`src/styles/input.css`, or you added new class names in a template	Run `npm run build:css`, then refresh. Or keep `npm run dev:css` running in a second terminal: it rebuilds on every save.
A file in `public/` (a script or picture)	Refresh. If the old version still shows, do a hard refresh: Ctrl + F5 (Windows/Linux) or Cmd + Shift + R (macOS).
`.env` or `package.json`	Stop and start. After a `package.json` change, run `npm install` first.

Looking at pages without a database

The project has a **preview server** that shows every page using made-up sample data. It is handy for design work, or if your database is not ready yet. Nothing is saved and forms do not submit.

```
node docs/ui-revamp/tools/preview-server.mjs
# then open http://localhost:4100/__pages   (Windows: or double-click docs/ui-revamp/tools/start-preview.bat)
```

Common setup errors

You see	What it means and the fix
`'node' is not recognized` / `command not found: node`	Node.js is not installed, or the terminal was open before you installed it. Install it, then open a new terminal.
`The database address is missing.`	No `.env`, or it is in the wrong folder, or the `*_DATABASE_URL` for your `DB_TARGET` is empty.
`Database not ready (ECONNREFUSED)`, then `Is Postgres started?`	PostgreSQL is not running. Start it (Windows: Services app → "postgresql" → Start; macOS: open Postgres.app; Linux: `sudo service postgresql start`).
`(28P01)` "password is wrong"	The password inside the database address is wrong.
`database "seniorpadi" does not exist`	Do step 4.
`EADDRINUSE: address already in use :::3000`	The site is already running in another terminal. Stop that one, or set `PORT=3001` in `.env`.
Pages show with no styling	Run `npm run build:css`.
`npm install` fails on `bcrypt`	Usually an old Node version. Install the current Node LTS, delete `node_modules`, run `npm install` again.

THINGS TO KNOW

- `start-windows.bat` tries to copy a file called `.env.example` when `.env` is missing, but **the project has no `.env.example`**. On a fresh copy the batch file will open an empty Notepad. Create `.env` by hand as in step 5.
- The start-up message says uploads are "switched off" without Cloudinary, but `src/services/cloudinary.js` actually saves them on your computer instead (`public/uploads/`, `storage/ids/`). The warning is out of date, not the uploads.

What you learned

- Install Node.js, Git and PostgreSQL; clone; `npm install`; create `.env`; `npm run dev`.
- The app creates its own tables and first admin account.
- Ctrl + C stops it. `npm run dev` restarts by itself; CSS changes need `npm run build:css`.

TRY IT YOURSELF

Run `npm run seed:local`, start the site, and log in as the demo admin (phone `08000000001`, password `password123`). Then log out and sign up as a brand-new senior with your own made-up details.

CHAPTER 5

The folder and file map

Every file in the project, grouped by what it is for, with one plain sentence each. Use this chapter like a map of a building: find the room first, then open the door.

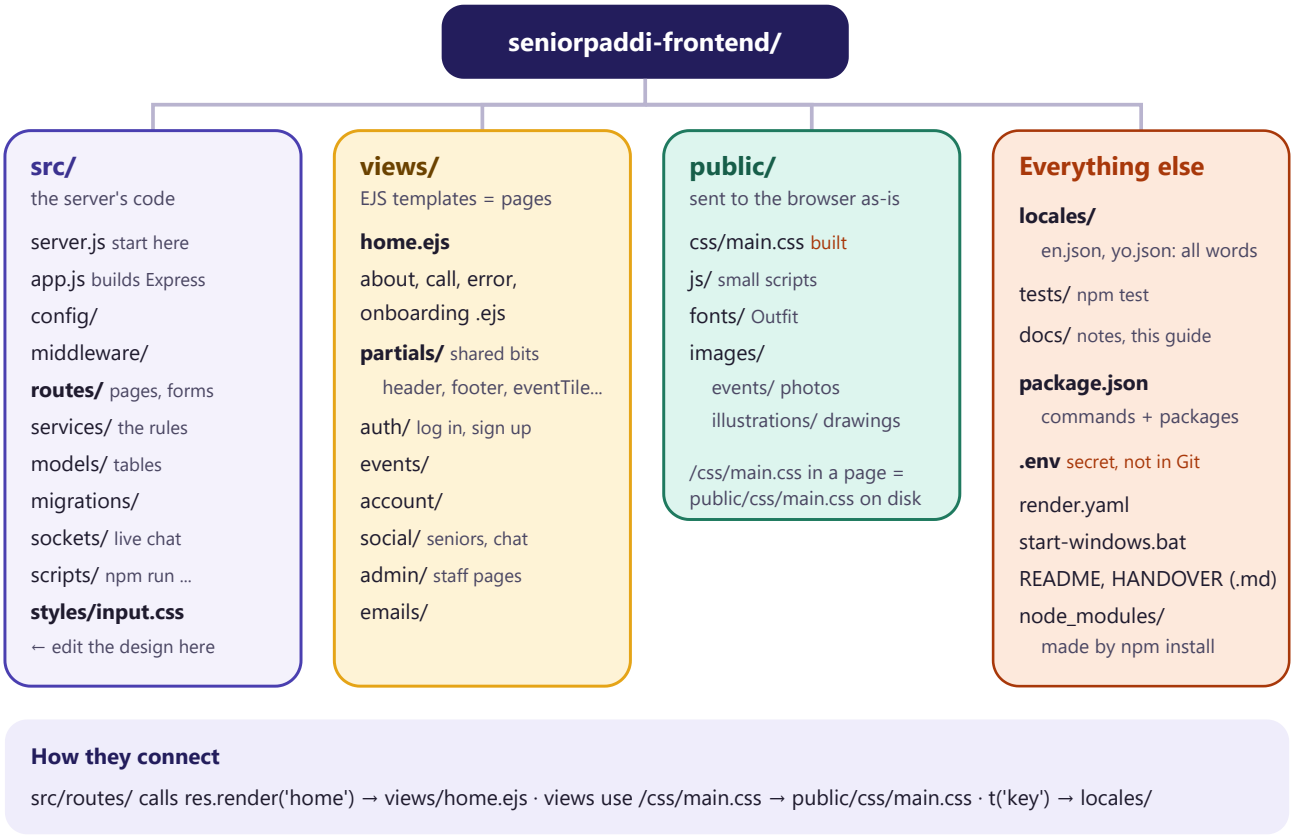


Figure 5.1 The four main areas of the project and how they connect.

The rule of thumb

If you want to change...	Look in
The words on a page	`locales/en.json` and `locales/yo.json`
The layout or HTML of a page	`views/` (the page) and `views/partials/` (shared pieces)
Colours, fonts, spacing, card styles	`src/styles/input.css`, then `npm run build:css`
What happens when a page opens or a form is sent	`src/routes/`
A rule used in several places (seats, SMS, dates)	`src/services/`

If you want to change...	Look in
What is stored in the database	`src/models/` and a new file in `src/migrations/`
Pictures	`public/images/` (and record them in `IMAGE_CREDITS.md`)
A small in-browser behaviour (pop-ups, password eye)	`public/js/`

IMPORTANT

There is no `controllers/` folder in this project. In many Express projects the code that answers a request lives in "controllers". Here it lives directly inside the route files in `src/routes/`, and shared logic lives in `src/services/`. When other tutorials say "controller", read "the function inside `router.get(...)` or `router.post(...)`".

Every file, grouped

This list is produced automatically from Git's own list of project files (`git ls-files`), so nothing is left out. The two entries in brackets are not stored in Git on purpose, but you will see them on your computer.

Project root: settings, starters and notes

`.gitignore`	Tells Git which files to leave out: `node_modules/`, `.env`, logs, `.vscode/` and any folder called `images/`.
`.sequelizerc`	Tells the Sequelize command-line tool where the config, models and migrations live (inside `src/`).
`DESIGN-NOTES.md`	Why the site looks the way it does: the design references, palette, contrast checks and what changed in each redesign pass.
`HANDOVER.md`	A no-code guide for the person who runs the live site day to day (Render, Neon, Brevo, Cloudinary, the admin pages).
`IMAGE_CREDITS.md`	Every picture used, where it is used, why it fits, its licence status, and a shopping list for better photos.
`README.md`	The project's front page on GitHub: what SeniorPadi does, how to run it, the settings table and the deployment plan.
`package-lock.json`	The exact version of every package that was installed, so every computer gets the same ones. Written by npm; don't edit by hand.
`package.json`	The project's ID card: its name, the `npm run` commands (scripts) and the list of packages it needs.

`render.yaml`	Instructions for Render (the hosting company): how to build and start the site and which settings to ask for.
`start-windows.bat`	Double-click starter for Windows: checks Node.js, installs packages the first time, then runs `npm start`.
`(.env)`	**Your private settings** (database address, passwords, keys). Not in Git on purpose: you create it yourself. See chapter 4.
`(node_modules/)`	Downloaded packages, created by `npm install`. Never edit it and never commit it; delete it and run `npm install` again if it breaks.

Server start and settings: src/server.js, src/app.js, src/config/

`src/app.js`	**Builds the Express app**: chooses EJS, serves `public/`, adds the middleware and plugs in every route file.
`src/config/config.js`	Database settings for Sequelize and its command-line tool.
`src/config/database.js`	Picks which database to use (local, Neon, Render or test) and whether it needs SSL.
`src/config/session.js`	The login session: stored in the database, cookie lasts 30 days.
`src/server.js`	**The starting point.** Checks settings, connects to the database, updates it, then opens the website.

Middleware: src/middleware/ (runs before every route)

`src/middleware/auth.js`	Gatekeepers: must be logged in, must be staff, must have changed a reset password, must be ID-verified.
`src/middleware/csrf.js`	Adds a secret token to every form and rejects forms without it (protects against fake forms on other sites).
`src/middleware/locals.js`	Runs on every request: finds the logged-in user, the language, translations and flash messages for the views.
`src/middleware/numericId.js`	Turns addresses like `/events/abc` into a friendly "not found" page.
`src/middleware/rateLimit.js`	Limits how often someone can try to log in, sign up, reset a password, upload or send messages.
`src/middleware/upload.js`	Receives uploaded pictures, ID documents and videos (in memory) and checks their type and size.

Routes: src/routes/ (one function per address)

`src/routes/account.js`	My account, my registrations (and cancelling), history, settings, profile picture, delete account.

`src/routes/admin.js`	**Every staff page** under `/admin`: dashboard, events, organizations, attendance, seniors, IDs, reports, staff, system.
`src/routes/events.js`	Find events (one card at a time), one event's page, Register, Volunteer, the "you're registered" pop-up, Notify me.
`src/routes/onboarding.js`	The welcome video shown once after signing up.
`src/routes/public.js`	**Pages anyone can open**: Home, About, Call, language switch, sign up, log in/out, forgot/reset/change password, /health.
`src/routes/social.js`	Find seniors, ID upload, skip, report, block, messages, phone exchange and notifications.
`src/routes/verify.js`	Confirm your phone (and email) with a 6-digit code before joining events or messaging.

Services: src/services/ (shared rules)

`src/services/attendance.js`	Records who came to an event (list, QR scan or ticket number all end here).
`src/services/audit.js`	Writes one line to the audit log, and never crashes a request.
`src/services/capacity.js`	Counts seats and registers people safely so an event can never be overbooked.
`src/services/cloudinary.js`	Uploads pictures, IDs and videos to Cloudinary and makes signed links to view IDs.
`src/services/cron.js`	Scheduled jobs: reminder texts at 8am, and marking finished events as completed every hour.
`src/services/dates.js`	Shows dates and times in Lagos time and converts form dates.
`src/services/i18n.js`	The `t()` function that looks up a sentence in English or Yorùbá.
`src/services/messaging.js`	Sends a chat message (used by the normal form and by live chat, so the rules match).
`src/services/migrate.js`	Applies new database migrations when the app starts and recreates missing tables.
`src/services/notify.js`	Sends SMS and email through Brevo (or prints them in the terminal when Brevo isn't set up).
`src/services/phone.js`	Checks and tidies Nigerian phone numbers (08031234567 → +2348031234567).
`src/services/phoneExchange.js`	The rules for asking to swap phone numbers and answering.
`src/services/qr.js`	Makes the optional QR check-in code picture.

`src/services/registration.js`	Registers a senior and sends the confirmation text/email; also `safeNext` for safe redirects.
`src/services/removal.js`	The two ways to remove a senior (from Find seniors only, or delete the account).
`src/services/social.js`	Rules for who can see and message whom (verified, not blocked, not skipped).
`src/services/startup.js`	Start-up checks: settings, waiting for the database, the welcome text, the first admin, the stylesheet.
`src/services/translate.js`	Translates event text into Yorùbá with Google Translate (optional) and caches it.
`src/services/verification.js`	Creates and checks the 6-digit confirmation codes.

Models: src/models/ (database tables)

`src/models/attendance.js`	Who actually came to an event.
`src/models/auditLog.js`	A record of important staff actions and failed messages.
`src/models/block.js`	Blocks between two seniors (both ways).
`src/models/event.js`	The Events table: title, description, dates, place, category, capacity, volunteers, status.
`src/models/eventInterest.js`	"Tell me when a similar event is listed" requests.
`src/models/index.js`	**Connects to the database**, loads every model and says how tables relate (an event belongs to an organization...).
`src/models/message.js`	Chat messages between two seniors.
`src/models/notification.js`	In-app notifications (new message, ID result, staff alerts).
`src/models/onboardingVideo.js`	The welcome video and its written transcript, one per language.
`src/models/organization.js`	The Organizations table: the churches, clubs and centres that host events.
`src/models/passwordReset.js`	One-time password-reset links (only a hash of the link is stored).
`src/models/phoneExchange.js`	"Can we swap phone numbers?" requests between two seniors.
`src/models/registration.js`	The Registrations table: who is going to which event, as attendee or volunteer, with a ticket number.
`src/models/report.js`	Reports a senior made about someone, for staff to review.
`src/models/reportHide.js`	Hides a reported senior from the person who reported them.

`src/models/skip.js`	Seniors someone skipped in Find seniors, so they don't show again.
`src/models/translation.js`	Saved Google Translate results, so each text is only translated once.
`src/models/user.js`	The Users table: name, phone, email, password hash, role (senior or admin), language, ID status and more.
`src/models/verificationCode.js`	The 6-digit codes sent to confirm a phone number or email.

Migrations: src/migrations/ (changes to the tables, in date order)

`src/migrations/20260927000001-create-users.js`	Migration: creates the Users table.
`src/migrations/20260927000002-create-organizations.js`	Migration: creates the Organizations table.
`src/migrations/20260927000003-create-events.js`	Migration: creates the Events table.
`src/migrations/20260927000004-create-registrations.js`	Migration: creates the Registrations table.
`src/migrations/20260927000005-create-event-interests.js`	Migration: creates the EventInterests table.
`src/migrations/20260927000006-create-password-resets.js`	Migration: creates the PasswordResets table.
`src/migrations/20260927000007-create-audit-logs.js`	Migration: creates the AuditLogs table.
`src/migrations/20260928000001-create-social-tables.js`	Migration: creates the messaging, skip, block, report and notification tables.
`src/migrations/20260929000001-phase3.js`	Migration: creates the OnboardingVideos table and adds onboardingSeenAt (Users) and qrToken (Registrations).
`src/migrations/20261003000001-events-tickets-attendance-verification.js`	Migration: adds ticket numbers, phone/email confirmation dates and reminder switches, plus the Attendances, PhoneExchanges, VerificationCodes and Translations tables.

Live chat and command-line scripts: src/sockets/, src/scripts/

`src/scripts/checkDb.js`	`npm run db:check`: says which database the app will use and whether it can reach it.
`src/scripts/createAdmin.js`	`npm run create-admin`: makes a staff account from the command line.
`src/scripts/seed.js`	`npm run seed`: wipes the database and fills it with demo events and users.

`src/scripts/translateUi.js`	`npm run translate:ui` : fills missing Yorùbá text in locales/yo.json with Google Translate.
`src/sockets/index.js`	Live chat with Socket.IO: only verified seniors, same login session as the website.

The stylesheet you edit: src/styles/

`src/styles/input.css`	**The stylesheet you edit**: Tailwind setup, brand colours (`@theme`), fonts and every shared class.

Views: top-level pages

`views/about.ejs`	About SeniorPadi.
`views/call.ejs`	Call SeniorPadi: the support phone number and hours.
`views/error.ejs`	The friendly error page (not found, not allowed, something went wrong).
`views/home.ejs`	**Home**: the hero, the three big choices, Events near you, How it works.
`views/onboarding.ejs`	The welcome video and transcript after signing up.

Views: shared pieces (partials)

`views/partials/authAside.ejs`	Side panel used on forgot, reset and change password pages.
`views/partials/confirmLogoutModal.ejs`	"Do you want to log out?" pop-up.
`views/partials/eventCard.ejs`	The event card used on Find events and the registered pop-up.
`views/partials/eventImage.ejs`	An event's picture: the uploaded photo, or the category cover photo.
`views/partials/eventTags.ejs`	Status tags like "Almost full" and "Closes in 3 days".
`views/partials/eventTile.ejs`	**The tall photo event card** used in Home's "Events near you" (dark and light versions).
`views/partials/flash.ejs`	Shows the one-time green/red/blue message ("Welcome, Ade!").
`views/partials/footer.ejs`	**Bottom of every page**: closes <main>, the dark footer, the log-out pop-up and closing scripts.
`views/partials/header.ejs`	**Top of every page**: <head>, styles, scripts, the one-line header and the opening of <main>.
`views/partials/langToggle.ejs`	The button that switches to the other language.
`views/partials/loginModal.ejs`	The log-in / sign-up pop-up when a guest taps Register.

`views/partials/nav.ejs`	The staff sidebar menu.
`views/partials/phoneInput.ejs`	A phone number box (numbers only).
`views/partials/qr.ejs`	The optional QR check-in code.
`views/partials/seniorNav.ejs`	The five-item menu (Home, Events, Seniors, Messages, My account), on top or at the bottom on phones.
`views/partials/ui/passwordInput.ejs`	A password box with a lock icon and a show/hide eye button.
`views/partials/ui/textInput.ejs`	A text box with an icon on the left.

Views: log in and account pages (auth/)

`views/auth/change.ejs`	Change password (forced after staff reset it).
`views/auth/forgot.ejs`	Forgot password.
`views/auth/login.ejs`	**Log in** ("Welcome Back" design).
`views/auth/reset.ejs`	Choose a new password from an email link.
`views/auth/signup.ejs`	**Create account**.
`views/auth/verify.ejs`	Confirm your phone (and email) with a 6-digit code.

Views: events/

`views/events/confirmed.ejs`	Confirmation page with the optional QR code.
`views/events/index.ejs`	**Find events**: one event card at a time with Register and Skip.
`views/events/registered.ejs`	The "you're registered" pop-up (with the volunteer offer).
`views/events/show.ejs`	One event's full page.

Views: account/

`views/account/account.ejs`	My account (the hub for everything personal).
`views/account/delete.ejs`	Delete my account.
`views/account/history.ejs`	Events I went to before.
`views/account/registrations.ejs`	My events: tickets and Cancel buttons.
`views/account/settings.ejs`	Settings: details, language, reminders, profile picture.

Views: social/ (Find seniors and messages)

`views/social/block.ejs`	Block a senior.
`views/social/find.ejs`	Find seniors: one senior at a time, or the ID upload if not verified yet.
`views/social/inbox.ejs`	The list of conversations.
`views/social/notifications.ejs`	Notifications.
`views/social/report.ejs`	Report a senior.
`views/social/thread.ejs`	One conversation, with the phone-swap button.

Views: admin/ (staff pages)

`views/admin/attendance.ejs`	Staff: attendance list for an event.
`views/admin/dashboard.ejs`	Staff dashboard.
`views/admin/event-form.ejs`	Staff: add or edit an event.
`views/admin/events.ejs`	Staff: list of events.
`views/admin/ids.ejs`	Staff: ID checks queue.
`views/admin/onboarding-video.ejs`	Staff: upload the welcome video and edit its transcript.
`views/admin/orgFields.ejs`	The organization form fields, shared by the two pages above.
`views/admin/organization-edit.ejs`	Staff: edit an organization.
`views/admin/organizations.ejs`	Staff: list and add organizations.
`views/admin/report.ejs`	Staff: one report with the conversation.
`views/admin/reports.ejs`	Staff: list of reports.
`views/admin/scan.ejs`	Staff: result of scanning a QR ticket.
`views/admin/seniors.ejs`	Staff: find a senior, reset password, remove.
`views/admin/staff.ejs`	Staff: manage staff accounts.
`views/admin/system.ejs`	Staff: is the database, SMS and upload service working?

Views: emails/

`views/emails/layout.ejs`	The HTML wrapper for every email (used by src/services/notify.js).

Public files: public/ (sent to the browser as they are)

`public/css/main.css`	The finished stylesheet the browser downloads. **Built from `src/styles/input.css`; never edit it by hand.**
`public/fonts/README.txt`	Where the Outfit font comes from and its licence.
`public/fonts/outfit-variable.woff2`	The Outfit font file (all weights in one file), served from our own site.
`public/images/README.md`	Notes on the pictures in this folder (some of it is older than IMAGE_CREDITS.md; trust IMAGE_CREDITS.md first).
`public/images/events/*` (12 files)	Cover photo for each event category (fitness, faith, learning, skills, social, health), as JPG and smaller WebP.
`public/images/hero.png`	An older hero picture. Nothing links to it any more.
`public/images/illustrations/*` (9 files)	Small original SVG drawings (elders chatting, a calendar, a phone and so on) used on several pages.
`public/js/chat.js`	Makes new chat messages appear without reloading (uses Socket.IO).
`public/js/dashboard.js`	Refreshes the numbers on the staff dashboard every 20 seconds.
`public/js/dialog.js`	Keyboard and screen-reader help for pop-ups (log in, "you're registered", requests).
`public/js/otp.js`	The six code boxes and the resend countdown on the Confirm-your-phone page.
`public/js/phone-input.js`	Keeps phone boxes to numbers only, 11 digits.
`public/js/swipe.js`	Swipe cards on Find events and Find seniors: drag right for yes, left to skip.
`public/js/top-loader.js`	The thin progress bar at the top while the next page loads.
`public/js/ui.js`	Shared page helpers: pop-ups, the show/hide password eye, busy buttons, confirm-password check, log-out question.
`public/js/upload.js`	Changes an upload button's text while a file is being sent.

Words: locales/

`locales/en.json`	**Every English sentence on the site**, each with a key like `"login.submit"`. Change wording here.
`locales/yo.json`	The same keys in Yorùbá. If a key is missing here, the English one is shown.

Tests: tests/

`tests/capacity.test.js`	Tests that events can never be overbooked, even with many people at once.
`tests/features.test.js`	Tests tickets, confirming a phone, swapping numbers, attendance, the dashboard, filters, reminders and translation.
`tests/helpers.js`	Shared helpers for the tests (make a test senior, a test event, log in).
`tests/newui.test.js`	Tests the newer screens (Find events cards, Skip, the registered pop-up).
`tests/phase3.test.js`	Tests live chat (Socket.IO) safety, QR check-in and the one-time welcome guide.
`tests/phone.test.js`	Tests the phone-number checks.
`tests/routes.test.js`	Tests that pages open, forms work without JavaScript and seniors can't open staff pages.
`tests/social.test.js`	Tests messaging, blocking, reporting and ID rules.
`tests/startup.test.js`	Tests that the app can set up an empty database on its own.

Documents and design tools: docs/

`docs/MVP.md`	The original project write-up (the first version, its features and design screenshots).
`docs/ui-revamp/GUIDE.md`	The plan for the accessibility-first redesign (phases 0 to 3) and how the work was split up.
`docs/ui-revamp/agents/*` (8 files)	Short briefs for the helper roles used during the redesign (reviewer, accessibility, tests and so on). Not used by the app.
`docs/ui-revamp/audit.md`	Phase 0: a list of every page and the problems found before the redesign.
`docs/ui-revamp/design-spec.md`	Phase 3: the design rules every restyled page had to follow.
`docs/ui-revamp/evidence/palette-results.md`	Raw output of the colour-contrast checker.
`docs/ui-revamp/palette.md`	Phase 2: the chosen colour palette and its measured contrast.
`docs/ui-revamp/phase3-results.md`	Phase 3: before-and-after results of the full restyle.
`docs/ui-revamp/research.md`	Phase 1: design research (palettes, layouts, accessibility standards, the Yorùbá font test).
`docs/ui-revamp/tools/content-freeze.mjs`	Checks that a restyle did not change any words, links or form fields.

`docs/ui-revamp/tools/contrast.mjs`	Checks that text and background colours have enough contrast to read.
`docs/ui-revamp/tools/freeze-approved.json`	The list of word/field changes that were approved during the redesign (read by content-freeze.mjs).
`docs/ui-revamp/tools/make-illustrations.mjs`	Draws the SVG illustrations in `public/images/illustrations/`.
`docs/ui-revamp/tools/page-checks.js`	A script pasted into a browser to check a page's layout (tap-target sizes, overflow and so on).
`docs/ui-revamp/tools/preview-server.mjs`	**Preview server**: shows every page with made-up data and no database, at `http://localhost:4100/__pages`.
`docs/ui-revamp/tools/start-preview.bat`	Double-click starter for the preview server on Windows.

THINGS TO KNOW

- `.gitignore` contains the line `images/`, which tells Git to ignore **every** folder called `images`, including `public/images/`. The pictures already in Git stay tracked, but a **new** picture you add there will be silently left out of commits unless you use `git add -f` (chapter 10 shows how). If that line was only meant for `docs/images/`, change it to `/docs/images/`.
- `public/images/hero.png` is not used by any page.
- `public/images/README.md` is partly out of date (it says no WebP versions exist and that the hero photo has no description; both have since changed). `IMAGE_CREDITS.md` is the current record.

What you learned

- `src/` is the server, `views/` are the pages, `public/` goes to the browser untouched, `locales/` holds the words.
- Route files do the "controller" job in this project.
- Never edit `public/css/main.css` or anything in `node_modules/` by hand.

TRY IT YOURSELF

Open `views/home.ejs` and find the line that includes `partials/eventTile`. Then open that partial. You have just followed a connection between two files, which is exactly what chapter 7 does for every important file.

CHAPTER 6

EJS explained for beginners

Every page you see on SeniorPadi starts life as an EJS template in `views/`. Once you can read EJS, you can change almost any page.

What EJS is, and why it exists

A plain HTML file always shows the same thing. But SeniorPadi's pages change: your name appears in the greeting, the list of events comes from the database, an error message appears only when something went wrong, and every word can be English or Yorùbá.

EJS (Embedded JavaScript) solves this. An EJS file is ordinary HTML with a few special tags, the "holes". When a route asks for a page, the server runs the template, fills every hole with real data, and sends the finished HTML to the browser. The browser never sees the EJS tags.

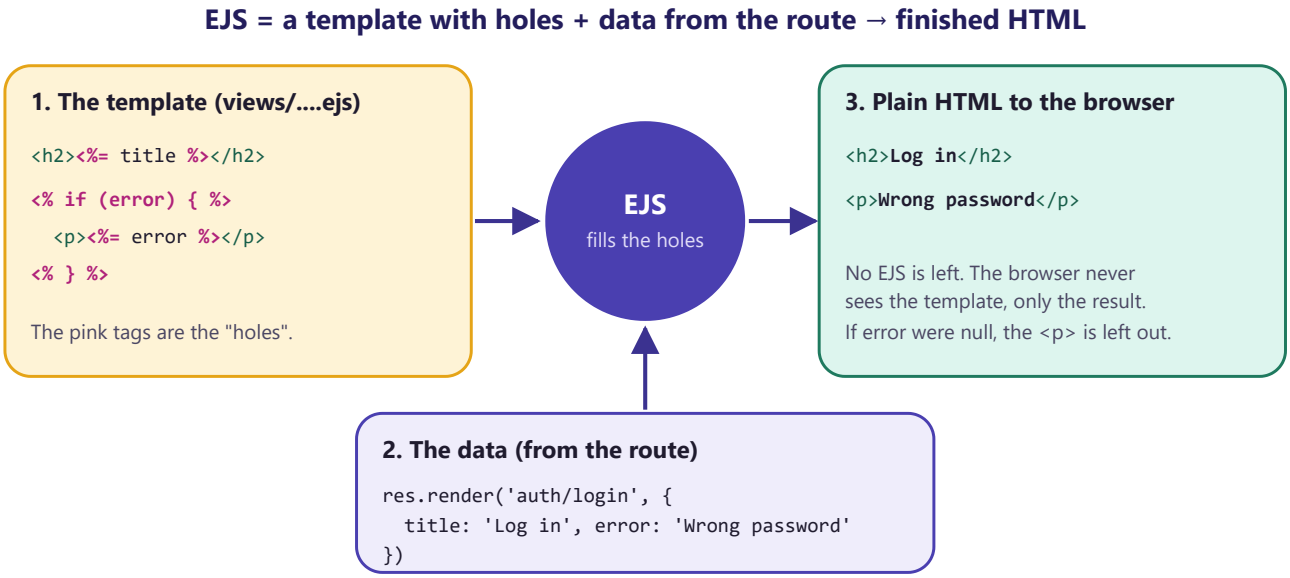


Figure 6.1 A template plus data makes HTML. This happens on the server for every page.

> **AN EVERYDAY PICTURE**
>
> An EJS template is like a printed form letter: "Dear ____, your appointment is on ____." The route is the clerk who fills in the blanks before posting it.

The four EJS tags used in this project

A count of every template in `views/` shows SeniorPadi uses exactly four kinds of tag. Each starts with `<%` and ends with `%>`.

Tag	Name	What it does	Used
`<%= x %>`	Output (escaped)	Prints the value of `x` into the page, made safe: characters like `<` are turned into harmless text.	917 times
`<%- x %>`	Output (unescaped)	Prints `x` exactly as it is, HTML and all. Used for `include(...)` and for HTML the server built itself.	145 times
`<% code %>`	Scriptlet	Runs JavaScript but prints nothing: `if`, loops, setting a variable.	442 times
`<%# note %>`	Comment	A note for people reading the template. It is removed completely and never reaches the browser.	95 times

1. Output, escaped: `<%= %>`

This is the tag you will use most. In the log-in page, the heading prints a sentence from the language files:

`views/auth/login.ejs`, line 21–22

```
<class="tk-tag">h1 class="mb-1 text-3xl md:text-4xl"><%= t('auth.welcome_back') %>
</class="tk-tag">h1>
<class="tk-tag">p class="mb-5 text-lg text-muted"><%= t('auth.welcome_sub') %></class="tk-
tag">p>
```

- `t('auth.welcome_back')` looks up that key in `locales/en.json` (or `yo.json`) and returns the sentence. `<%=` prints it.
- The class names (`text-3xl`, `text-muted`) are ordinary HTML and are styled by Tailwind (chapter 9).

> **IMPORTANT: WHY "ESCAPED" MATTERS**
>
> If a senior typed `<script>` as their name, `<%= %>` prints it as harmless text instead of running it. This stops a classic attack called *XSS* (cross-site scripting). **Always use `<%= %>` for anything a person typed.**

2. Output, unescaped: `<%- %>`

Use it only for HTML you trust, and above all for **includes** (next section). The QR code partial uses it to print an SVG picture that the server drew itself (`src/services/qr.js`):

`views/partials/qr.ejs`, line 5

```
<class="tk-tag">div class="mx-auto w-48 max-w-full" aria-hidden="true"><%- qr %>
</class="tk-tag">div>
```

3. Scriptlet: `<% %>`

Runs JavaScript without printing. The opening and closing braces of an `if` or a loop each sit in their own scriptlet, with ordinary HTML between them. Here the log-in page shows an error box only when there is an error:

views/auth/login.ejs, line 25–27

```
<% if (error) { %>
  <class="tk-tag">div role="alert" class="alert alert-error"><class="tk-tag">i class="fa-solid fa-circle-xmark mt-1 shrink-0" aria-hidden="true"></class="tk-tag">i><class="tk-tag">span><%= error %></class="tk-tag">span></class="tk-tag">div>
<% } %>
```

- `<% if (error) { %>`: "if there is an error, include the next lines".
- `<%= error %>`: print the message.
- `<% } %>`: the end of the `if`. Forgetting it is the most common EJS mistake.

Scriptlets can also set up variables at the top of a template. The event card works out which look to use:

views/partials/eventTile.ejs, line 8–10

```
<% const v = typeof variant !== 'undefined' && variant === 'light' ? 'light' : 'dark'; %>
<% const evTitle = event.tTitle || event.title; %>
<% const evText = event.tDescription || event.description || ''; %>
```

- `typeof variant !== 'undefined'` asks "was I given a `variant`?". If not, it uses `'dark'`.
- `event.tTitle || event.title`: use the Yorùbá title if there is one, otherwise the original.

4. Comment: `<%# %>`

views/auth/login.ejs, line 3

```
<%# "Welcome Back" layout (ref): mesh-gradient art panel | frosted form card + social-proof strip. Fits the screen from 1024px %>
```

Comments explain why a template is built the way it is. The project uses them a lot, so read them: they are often the best documentation.

How the server sends data to a view

A route calls `res.render(name, data)`. The **name** is a path inside `views/` without `.ejs`; the **data** is a JavaScript object whose keys become variables in the template.

src/routes/public.js, line 44

```
res.render('home', { title: 'SeniorPadi', fullWidth: true, upcoming });
```

- Opens `views/home.ejs`.
- Inside it, `title` is `'SeniorPadi'`, `fullWidth` is `true`, and `upcoming` is the list of events the route just loaded. (`{ upcoming }` is shorthand for `{ upcoming: upcoming }`.)

Some variables are available in **every** template without being passed. They are set by the middleware `src/middleware/locals.js` on `res.locals`, which Express merges into every render:

src/middleware/locals.js, line 20–28

```
res.locals.currentUser = user;
res.locals.unreadCount = user ? await Notification.count({ where: { userId: user.id,
readAt: null } }) : 0;
res.locals.lang = lang;
res.locals.t = req.t;
res.locals.formatDate = (d) => dates.formatDate(d, lang);
res.locals.formatTime = dates.formatTime;
res.locals.currentPath = req.path;
res.locals.supportPhone = process.env.SUPPORT_PHONE || '';
```

Variable	What it holds
`currentUser`	The logged-in person (a User from the database), or `null` for a visitor.
`unreadCount`	How many notifications they have not read (the badge on "My account").
`lang`	`'en'` or `'yo'`.
`t(key, vars)`	Looks up a sentence in the current language. `t('home.welcome_back', { name: 'Ade' })` fills `{name}`.
`formatDate(d)`, `formatTime(d)`	Show a date or time in Lagos time, in the right language.
`currentPath`	The address of this page, e.g. `/events` (used to highlight the menu).
`supportPhone`	The help line number from `SUPPORT_PHONE`.
`flash`	A one-time message such as "Welcome, Ade!", or `null`.
`csrfToken`	The secret form token (set by `src/middleware/csrf.js`). Every POST form needs it.

Following one real page: Log in

1. **The route** in `src/routes/public.js` answers `GET /login`:

src/routes/public.js, line 142–144

```
router.get('/login', (req, res) => {
  res.render('auth/login', { title: req.t('login.title'), error: null, phone: '',
after: afterAuth(req.query) });
});
```

2. **The data**: `title` (for the browser tab), `error: null` (nothing wrong yet), `phone: ''` (empty box) and `after` (where to go after logging in, e.g. straight into registering for an event).
3. **The template** `views/auth/login.ejs` starts by including the shared header and passing it one extra value:

views/auth/login.ejs, line 1

```
<%- include('../partials/header', { pageLayout: 'auth' }) %>
```

4. **The header** (`views/partials/header.ejs`) writes the `<head>` with `<%= title %> | SeniorPadi` as the tab title, loads the stylesheet and scripts, and, because `pageLayout` is `'auth'`, leaves out the usual menu bar.
5. **The body** of `login.ejs` prints the headings with `t(...)`, the error box only if `error` is set, the hidden `_csrf` field, and the phone and password boxes (each an include, see below).
6. **The footer** include closes the page. EJS joins everything into one HTML document and Express sends it.

When the form is sent with a wrong password, the POST route calls the same template again, but with `error` set to a message and `phone` set to what was typed. Same template, different data, different page.

Partials: reusing pieces

A **partial** is a small template meant to be placed inside other templates. SeniorPadi keeps them in `views/partials/`. You place one with `include`, always inside `<%- %>` (the dash, so its HTML is not escaped):

views/auth/login.ejs, line 48

```
<%- include('../partials/ui/passwordInput', { id: 'password', name: 'password',
autocomplete: 'current-password', required: true }) %>
```

- The path is relative to the current file: `login.ejs` is in `views/auth/`, so `../partials/` goes up one folder.
- The object `{ id: 'password', ... }` gives the partial its own variables. Inside `passwordInput.ejs`, `id` is `'password'`.

- A partial also sees everything the page sees (`t`, `currentUser` and so on).

The "layout": header and footer

Many frameworks have one layout file that wraps every page. SeniorPadi does the same job with two partials: **every** page starts with `include('partials/header')` and ends with `include('partials/footer')`. The header opens `<html>`, `<body>` and `<main>`; the footer closes them. That is why one change in `header.ejs` changes every page.

The header changes shape depending on what the page passes it:

views/partials/header.ejs, line 3–11

```
  const isAdmin = currentUser && currentUser.role === 'admin';
  const isWide = typeof wide !== 'undefined' && wide;
  const isFull = typeof fullWidth !== 'undefined' && fullWidth; // landing page: sections
go edge to edge
  // pageLayout 'auth': log in / create account fill the screen on their own (no site bar,
no bottom menu, no footer)
  // pageLayout 'hero': Home draws its own top bar inside the hero
  const layout = typeof pageLayout !== 'undefined' ? pageLayout : '';
  const isAuthPage = layout === 'auth';
  const ownTopbar = layout === 'hero';
%>
```

Page passes	Result
nothing	Normal page: one-line top bar, menu, narrow centred content.
`wide: true`	Wider content area (events, account pages).
`fullWidth: true`	Sections go edge to edge (Home).
`pageLayout: 'auth'`	Full-screen log-in / create-account: no top bar, no bottom menu, no footer.
`pageLayout: 'hero'`	Home draws its own top bar inside the hero.
(a staff member is logged in)	Staff sidebar instead of the senior menu.

Loops and if/else: the event cards

The "Events near you" section of Home shows one card per event, or a friendly message when there are none. It is a good example of `if/else` and a loop working together:

views/home.ejs, line 62–76

```
      <% if (upcoming.length > 0) { %>
        <class="tk-tag">span class="text-5xl font-bold tracking-tight md:text-6xl"><%=
upcoming.length %></class="tk-tag">span><class="tk-tag">span class="text-lg font-semibold">
```

```
<%= t('home.events_count') %></class="tk-tag">span>
      <% } else { %>
        <class="tk-tag">span class="text-5xl font-bold tracking-tight md:text-6xl"><%=
t('events.free') %></class="tk-tag">span>
      <% } %>
    </class="tk-tag">p>
    <class="tk-tag">div class="flex lg:justify-end">
      <class="tk-tag">a href="/events" class="btn-white"><%= t('nav.find_events') %>
<class="tk-tag">span class="btn-icon"><class="tk-tag">i class="fa-solid fa-arrow-right"
aria-hidden="true"></class="tk-tag">i></class="tk-tag">span></class="tk-tag">a>
    </class="tk-tag">div>
  </class="tk-tag">div>
</class="tk-tag">section>

<%# ================= THE THREE CHOICES (Eldora tiles, ref 8). Call uses the yolk accent
================= %>
<class="tk-tag">section class="mx-auto max-w-6xl px-4 pt-6 pb-6 md:pt-10" aria-
labelledby="what-title">
  <class="tk-tag">h2 id="what-title" class="mb-1"><%= t('home.what_kicker') %></class="tk-
tag">h2>
```

- `upcoming.length > 0`: are there any events?
- `upcoming.forEach(function (event, i) { ... })` repeats the `<li>` once per event. `event` is the current event; `i` counts 0, 1, 2, 3.
- `i % 2 === 0 ? 'dark' : 'light'`: even cards dark, odd cards light (`%` is "remainder after dividing").
- `<% } else { %>`: otherwise, show the empty-state picture and a button.

The flash message partial is a small, complete example of the same ideas:

views/partials/flash.ejs, line 1–11

```
<% if (flash) {
     const styles = {
       success: 'alert alert-success',
       error: 'alert alert-error',
       info: 'alert alert-info'
     };
%>
  <class="tk-tag">div role="<%= flash.type === 'error' ? 'alert' : 'status' %>" class="<%=
styles[flash.type] || styles.info %>">
    <class="tk-tag">i class="fa-solid <%= flash.type === 'success' ? 'fa-circle-check' :
(flash.type === 'error' ? 'fa-circle-xmark' : 'fa-circle-info') %> mt-1 shrink-0" aria-
hidden="true"></class="tk-tag">i>
    <class="tk-tag">span class="min-w-0"><%= flash.message %></class="tk-tag">span>
  </class="tk-tag">div>
```

Common beginner mistakes

Mistake	What happens, and the fix
Forgetting a closing `<% } %>`	The page fails with an error like `Unexpected token` or `missing ) after argument list`. Count your opening and closing braces.
Using a variable the route didn't pass	`ReferenceError: x is not defined`. Pass it in `res.render`, or check it with `typeof x !== 'undefined'` as the partials do.
`<%= include(...) %>` (equals instead of dash)	The page shows raw HTML code as text. Use `<%- include(...) %>`.
`<%- %>` on something a user typed	A security hole (XSS). Use `<%= %>`.
Wrong include path	`Could not find the include file`. Paths are relative to the current template.
Wrong name in `res.render`	`Failed to lookup view "auth/logn"`. The name is the path inside `views/` without `.ejs`.
Typing English text straight into a template	It works, but Yorùbá users see English. Add a key to both `locales/` files and use `t('your.key')`.
Expecting EJS code to run in the browser	It runs once, on the server, before the page is sent. For things that react to clicks, use a script in `public/js/`.
A new POST form with no `_csrf` field	"The form has expired" error. Add `<input type="hidden" name="_csrf" value="<%= csrfToken %>">`.

What you learned

- `<%= %>` prints safely, `<%- %>` prints raw HTML (for includes), `<% %>` runs code, `<%# %>` is a comment.
- `res.render('folder/name', { ... })` picks the template and gives it data; `res.locals` adds data to every page.
- Every page is header partial + its own content + footer partial.

TRY IT YOURSELF

In `views/partials/flash.ejs`, the icon for a success message is `fa-circle-check`. Change it to `fa-face-smile`, save, then create a new test account: the green "Welcome" message on the next page now has a smiley. Change it back afterwards. You just edited a partial that every page uses.

CHAPTER 7

File-by-file walkthrough

A reference for the important files: what each one does, what goes in and what comes out, which files it talks to, and the key lines in plain English. Small, similar files are grouped in tables.

> **HOW TO READ THE CONNECTION BOXES**
>
> **Used by** lists the files that load or call this one (if you change it, check those). **Uses** lists the files this one depends on. Nearly every server file also uses `src/models/index.js` to reach the database; that is only mentioned where it matters.

Starting up

`src/server.js`

What and why: the very first file that runs (`npm start` runs `node src/server.js`). It does all the set-up so nobody has to type set-up commands, then starts the web server. Chapter 2 walks through it.

Receives: settings from `.env`. **Sends:** nothing; it starts the server and prints progress in the terminal.

Used by	`package.json` scripts (`start`, `dev`), `start-windows.bat`
Uses	`src/services/startup.js`, `src/models/index.js`, `src/services/migrate.js`, `src/app.js`, `src/sockets/index.js`, `src/services/cron.js`

`src/app.js`

What and why: builds the Express app. The order of the lines here *is* the order every request is handled in, so this is the file to read to understand the request path.

`src/app.js`, line 18–29

```
app.use(express.static(path.join(__dirname, '../public'), { maxAge: isProduction ? '7d' : 0
}));
app.use(express.urlencoded({ extended: false }));

app.use(sessionMiddleware);

app.use(locals);
// token is created first so error pages can still show forms
```

```
app.use(addCsrfToken);
app.use(csrfProtection);
app.use(requirePasswordChanged);

app.use('/', require('./routes/public'));
```

- `express.static`: if the address matches a file in `public/`, send that file and stop.
- `express.urlencoded`: read submitted form fields into `req.body`.
- `sessionMiddleware`, `locals`, `addCsrfToken`, `csrfProtection`, `requirePasswordChanged`: the middleware, in order (next section).

After the routes come two "catch-all" handlers: a 404 page for addresses nobody answered, and an error page for anything that crashed (with a special message when a form's security token has expired).

Used by	`src/server.js`; every test file in `tests/` (they load the app without starting a server)
Uses	`src/config/session.js`, `src/middleware/*`, every file in `src/routes/`, `views/error.ejs`

`src/config/database.js`

What and why: decides *which* database to use and whether it needs an encrypted (SSL) connection. Keeping it in one place means Sequelize, the migration tool and the session store always talk to the same database.

`src/config/database.js, line 5–8`

```
//   DB_TARGET=local  -> LOCAL_DATABASE_URL (Postgres on your computer)
//   DB_TARGET=neon   -> NEON_DATABASE_URL  (Neon in the cloud)
//   no DB_TARGET     -> DATABASE_URL       (what Render sets in production)
//   running tests    -> TEST_DATABASE_URL  (always, it gets wiped)
```

Used by	`src/config/config.js`, `src/config/session.js`, `src/scripts/checkDb.js`, `src/scripts/seed.js`
Uses	the `.env` settings

`src/config/config.js` and `.sequelizerc`

`config.js` turns the result of `database.js` into Sequelize settings (always store times in UTC, small connection pool). `.sequelizerc` tells the Sequelize CLI where `config.js`, the models and the migrations are.

Used by	`src/models/index.js`, the `sequelize-cli` commands (`npm run db:migrate`)
Uses	`src/config/database.js`

`src/config/session.js`

What and why: sets up login sessions, stored in a `session` table in the same database (created automatically). It lives in its own file because both Express and live chat need the very same session.

Used by	`src/app.js`, `src/sockets/index.js`
Uses	`src/config/database.js`, packages `express-session`, `connect-pg-simple`, `pg`

`src/services/startup.js and src/services/migrate.js`

`startup.js` holds the set-up steps `server.js` calls: check settings (and warn about missing optional ones), wait for the database, add the default welcome-guide text, create the first admin from `ADMIN_*` settings, and rebuild the stylesheet if it is missing. `migrate.js` runs any migration files that have not run yet (it records finished ones in a table called `SequelizeMeta`) and recreates any table that has gone missing. It never deletes data.

`src/services/startup.js`, line 113

```
if (!phone || !password || password.length < 8) return 'missing';
```

→ The first admin is only created if the phone number is valid and the password has at least 8 characters.

Used by	`src/server.js` (both); `migrate.js` also by `src/routes/admin.js` (System page), `src/scripts/seed.js`, `src/scripts/createAdmin.js`
Uses	`src/services/phone.js`, `locales/*.json`, `src/migrations/*`

Middleware

`src/middleware/locals.js`

What and why: runs on every request. It looks up the logged-in user from the session, picks the language, and puts shared things on `res.locals` so every template can use them (`currentUser`, `t`, `formatDate`, `flash`…). It also creates `req.flash()` for one-time messages.

`src/middleware/locals.js`, line 14–18

```
const lang = user ? user.preferredLanguage : (req.session.lang || 'en');

req.user = user;
req.lang = lang;
req.t = (key, vars) => t(key, lang, vars);
```

→ A logged-in person's language comes from their account; a visitor's from their session; otherwise English.

→ `req.t(key)` is the translation function routes use; templates get the same function as `t`.

Used by	`src/app.js` (so, every page)
Uses	`src/models` (User, Notification), `src/services/i18n.js`, `src/services/dates.js`

`src/middleware/auth.js`

What and why: four "gatekeepers" that routes put in front of pages:

Function	Lets you through only if...
`requireAuth`	you are logged in (otherwise: remembers the page and sends you to `/login`).
`requireAdmin`	you are staff (otherwise: "not allowed" page).
`requirePasswordChanged`	you don't need to change a password a staff member reset.
`requireVerified`	you are a senior whose ID was approved (for Find seniors and messages).

Used by	`src/app.js` and every file in `src/routes/`
Uses	`views/error.ejs`

`src/middleware/csrf.js`

Creates the secret form token (`csrfToken` in templates) and rejects POST requests without it. Upload forms send the token in the address instead, because the file has not been read yet when the check runs.

Used by	`src/app.js`
Uses	package `csrf-sync`

The other middleware

File	What it does	Used by
`src/middleware/rateLimit.js`	Five limiters: log in, sign up, password reset, uploads, messages. Too many tries shows a friendly "please wait" page. Switched off during tests.	`routes/public.js`, `account.js`, `social.js`
`src/middleware/numericId.js`	If an address like `/events/:id` has letters instead of a number, shows "not found" before touching the database.	`routes/events.js`, `account.js`, `social.js`, `admin.js`

File	What it does	Used by
`src/middleware/upload.js`	Receives one uploaded file into memory and checks its type and size (pictures 2–3 MB, IDs 5 MB, videos 50 MB). Problems become friendly messages in `req.uploadError`.	`routes/account.js`, `social.js`, `admin.js`

Routes

Each route file creates a `router`, attaches functions to addresses with `router.get` or `router.post`, and is plugged into the app in `src/app.js`. Inside each function, `req` is the request (what came in) and `res` is the response (what goes back).

`src/routes/public.js`

What: the pages anyone can open, plus everything about accounts and passwords.

Address	What happens
`GET /health`	Checks the database; answers `{"ok":true}`.
`GET /`	Home. Staff are sent to `/admin`. Loads the next 4 events; renders `home`.
`GET /about`, `GET /call`	About and Call pages.
`POST /language`	Switches English/Yorùbá and goes back to the same page.
`GET/POST /signup`	Create account (chapter 8).
`GET/POST /login`, `POST /logout`	Log in (chapter 8) and log out (ends the session, goes to `/login`).
`/forgot-password`, `/reset-password/:token`, `/change-password`	Password help. A reset link is only emailed if the account has an email; otherwise staff reset it.

Used by	`src/app.js`
Uses	models User, PasswordReset, Event, Organization; services `translate`, `phone`, `registration`, `notify`; middleware `auth`, `rateLimit`; views `home`, `about`, `call`, `auth/*`, `error`

`src/routes/events.js`

What: finding and joining events.

Address	What happens
`GET /events`	Find events: shows *one* open event at a time (optionally one category), skipping full ones and ones you already joined.
`POST /events/:id/skip`, `POST /events/start-again`	Skip a card (remembered in the session), or clear the skips.
`GET /events/:id`	One event's page, with its tags and your status.
`POST /events/:id/register`, `/volunteer`	Take a place (log-in and a confirmed phone required).
`GET /events/:id/registered`, `/confirmed`	"You're registered" pop-up with the ticket number; confirmation with the QR code.
`POST /events/:id/notify-me`	"Tell me when a similar event is listed".

Used by	`src/app.js`
Uses	services `capacity`, `registration`, `translate`, `qr`; `src/routes/verify.js` (`needsContactCheck`); views `events/*`

`src/routes/verify.js`

Confirming a phone number (and email, if given) with a 6-digit code before joining an event or messaging. It remembers what you were trying to do in `req.session.afterVerify` and carries on once you are confirmed.

Used by	`src/app.js`, `src/routes/events.js`
Uses	`src/services/verification.js`, `src/services/registration.js`, `views/auth/verify.ejs`, `public/js/otp.js` (in the page)

`src/routes/account.js`

My account (`/account`), my tickets (`/registrations`, with Cancel), past events (`/history`), settings and profile picture (`/settings`), and deleting the account (`/account/delete`). All need a log-in.

Used by	`src/app.js`
Uses	services `notify`, `qr`, `cloudinary`, `removal`, `dates`; middleware `upload`, `rateLimit`; views `account/*`

`src/routes/social.js`

Find seniors (one at a time; or the ID upload if not yet verified), skip, report, block, the message inbox and conversations, swapping phone numbers, and notifications. Almost everything here is behind `requireVerified`.

Used by	`src/app.js`
Uses	services `social`, `messaging`, `phoneExchange`, `translate`, `cloudinary`; views `social/*`

`src/routes/onboarding.js`

The welcome video shown once after signing up (`/onboarding`), always skippable, with a written transcript. `POST /onboarding/done` records that it was seen.

Used by	`src/app.js`
Uses	model OnboardingVideo, `src/services/cloudinary.js`, `views/onboarding.ejs`

`src/routes/admin.js`

The largest file (about 780 lines): every staff page under `/admin`. Its first lines put `requireAdmin` in front of *all* of them:

`src/routes/admin.js`, line 22

```
router.use(requireAdmin);
```

Pages: dashboard (`/admin`, with live numbers from `/admin/stats.json`), events (add, edit, cancel, attendance, QR check-in), organizations, seniors (search, reset password, remove), ID checks, reports, the welcome video, staff accounts and the System status page.

Used by	`src/app.js` (at `/admin`)
Uses	services `attendance`, `capacity`, `cloudinary`, `notify`, `audit`, `dates`, `removal`, `social`, `phone`, `migrate`, `cron`; views `admin/*`

Services (the rules)

`src/services/capacity.js`

What and why: everything about seats. Its `register()` function makes sure an event can never be overbooked, even if two people press Register for the last seat at the same moment. It does this with a database *transaction* (a group of steps that all happen or none do) and a *lock* on the event

row (others wait their turn).

src/services/capacity.js, line 57–58

```
return sequelize.transaction(async (transaction) => {
  const event = await Event.findByPk(eventId, { transaction, lock: transaction.LOCK.UPDATE
});
```

It also works out the tags shown on events ("Almost full" when 5 or fewer places are left, "Closes in X days" when 5 or fewer days are left) in `eventState()`.

Used by	`src/services/registration.js`, `src/routes/events.js`, `src/routes/admin.js`; tested by `tests/capacity.test.js`
Uses	models Event, Registration; `src/services/dates.js`

`src/services/registration.js`

Calls `capacity.register()`, turns a failure reason into a message key (`full`, `closed`, `already`), and on success sends the confirmation SMS/email with the ticket number. Also has `safeNext()`, which only allows redirects to SeniorPadi's own pages.

Used by	`src/routes/events.js`, `src/routes/public.js`, `src/routes/verify.js`
Uses	`capacity.js`, `notify.js`, `dates.js`

`src/services/notify.js`

Sends email and SMS through Brevo. The words come from `locales/` (keys ending in `.sms`, `.email`, `.subject`) in the person's language; emails are wrapped in `views/emails/layout.ejs`. In "dry run" mode (no Brevo key, or `NOTIFY_DRY_RUN=true`) messages are printed in the terminal instead. A failed message is logged and never stops a registration.

Used by	routes `public`, `account`, `admin`; services `registration`, `cron`, `removal`, `verification`
Uses	`i18n.js`, `audit.js`, `views/emails/layout.ejs`

The other services

File	What it does	Used by
`i18n.js`	`t(key, lang, vars)`: finds the sentence, falls back to English, fills in `{name}`-style gaps.	`middleware/locals.js`, `notify.js`
`dates.js`	Formats dates/times in Lagos time; converts form date boxes.	locals, routes, many services

File	What it does	Used by
`phone.js`	`normalizePhone()` turns 0803... into +234803...; `phoneProblem()` says what is wrong.	`routes/public.js`, `admin.js`, `startup.js`, `createAdmin.js`
`verification.js`	Makes, sends and checks 6-digit codes.	`routes/verify.js`
`translate.js`	Yorùbá for event text via Google; results cached in the Translations table.	routes `public`, `events`, `social`
`cloudinary.js`	Uploads and deletes files; signed (private) links for IDs; local saving when not set up.	routes `account`, `admin`, `onboarding`, `social`
`cron.js`	8am reminders; hourly "mark finished events completed".	`server.js`, `admin.js` (System page)
`social.js`	Who can see/message whom; in-app notifications.	`routes/social.js`, `admin.js`, sockets, messaging
`messaging.js`	Saves and sends one chat message with the same rules for form and live chat.	`routes/social.js`, `sockets/index.js`
`phoneExchange.js`	Ask to swap numbers; the number shows only after Accept.	`routes/social.js`
`removal.js`	Remove from Find seniors only, or delete the account (keeps registrations honest).	`routes/account.js`, `admin.js`
`attendance.js`	Mark someone as came / didn't come; list, QR and ticket number all end here.	`routes/admin.js`
`qr.js`	Draws the QR code SVG for a registration.	`routes/events.js`, `account.js`
`audit.js`	Writes one line to the audit log; never crashes a request.	`admin.js`, `notify.js`, `removal.js`

Models (the database tables)

`src/models/index.js`

What and why: connects to the database, loads all 18 models, and declares how tables relate (an Event *belongs to* an Organization; a User *has many* Registrations). Everything else gets models from here: `const { User, Event } = require('../models')`.

src/models/index.js, line 28–32

```
Organization.hasMany(Event, { foreignKey: 'organizationId' });
Event.belongsTo(Organization, { foreignKey: 'organizationId' });

Event.hasMany(Registration, { foreignKey: 'eventId' });
Registration.belongsTo(Event, { foreignKey: 'eventId' });
```

Used by	almost every file in `src/`, plus the tests and scripts
Uses	`src/config/config.js`, every other file in `src/models/`

src/models/user.js

The Users table. Seniors and staff are both users; `role` tells them apart. `paranoid: true` means deleting a user only marks them deleted (a "soft delete"), so history stays correct. It adds three helpers you will see in templates and routes:

src/models/user.js, line 34–46

```
User.prototype.contactVerified = function () {
  return Boolean(this.phoneVerifiedAt) && (!this.email || Boolean(this.emailVerifiedAt));
};

// 08031234567 instead of +2348031234567
User.prototype.localPhone = function () {
  return this.phone ? '0' + this.phone.replace('+234', '') : '';
};

// first name only, for friendly greetings
User.prototype.firstName = function () {
  return this.fullName.split(' ')[0];
};
```

Used by	`src/models/index.js` (and through it, everything)
Uses	Sequelize; the table itself is created by `src/migrations/20260927000001-create-users.js` and later migrations

src/models/event.js and src/models/registration.js

`Event` holds what an event is (title, description, picture, start/end, place, `category`, `capacity`, volunteers, deadline, `status`). Its six categories are listed once and reused everywhere: `fitness`, `faith`, `learning`, `skills`, `social`, `health`.

`Registration` is one person's place at one event, as `attendee` or `volunteer`, with a status (`registered`, `cancelled`, `attended`, `no_show`), a QR token and a ticket number like `SP-7K3QX9` (no 0/O or 1/I/L, so it can be read out on the phone).

Used by	`src/models/index.js`; heavily by `routes/events.js`, `services/capacity.js`, `routes/admin.js`
Uses	Sequelize

The other models

Model (file)	One row is...
Organization	a church, club or centre that hosts events.
EventInterest	"tell me about similar events" for one category.
PasswordReset	a one-hour reset link (only its hash is stored).
VerificationCode	a 6-digit code for a phone or email.
Message, Notification	a chat message; an in-app notification.
Skip, Block, Report, ReportHide	Find-seniors safety: skipped, blocked, reported, hidden after a report.
PhoneExchange	a request to swap phone numbers.
Attendance	a person who actually came to an event.
OnboardingVideo	the welcome video and transcript for one language.
Translation	a saved Google translation.
AuditLog	a record of a staff action or a failed message.

Views (the pages)

`views/partials/header.ejs and views/partials/footer.ejs`

What and why: the top and bottom of every page (chapter 6 explains the layout options). The header loads the font, the icons, `/css/main.css?v=ui5` and three scripts; draws the skip link, the thin loading bar, the one-line top bar and the menu; and opens `<main>`. The footer closes `<main>`, draws the dark footer with four links, adds the log-out pop-up for logged-in people and loads two more scripts.

Used by	every page in `views/`
Uses	partials `seniorNav`, `nav` (staff), `langToggle`, `flash`, `confirmLogoutModal`; `public/css/main.css`; `public/js/phone-input.js`, `top-loader.js`, `ui.js`, `swipe.js`, `dialog.js`

views/auth/login.ejs

The "Welcome Back" log-in page: on large screens a colourful art panel on the left and a frosted card with the form on the right, with a small trust strip under it. It is a full-screen page (`pageLayout: 'auth'`).

Receives: `error`, `phone`, `after` from the route. **Sends:** `POST /login` with `_csrf`, `registerEventId`, `next`, `phone`, `password`.

views/auth/login.ejs, line 29

```
<class="tk-tag">form action="/login" method="post" class="form-soft form-compact" data-busy-text="<%= t('auth.logging_in') %>">
```

→ `data-busy-text` is read by `public/js/ui.js` to show "Logging you in..." with a spinner while the page loads.

Used by	`src/routes/public.js` (GET and POST `/login`)
Uses	partials `header`, `footer`, `langToggle`, `flash`, `phoneInput`, `ui/passwordInput`; CSS classes `auth-frame`, `auth-art`, `auth-card`, `btn-glow`

views/auth/signup.ejs

"Create your account": the form on a white panel with a warm beige picture panel beside it. Fields, in order: full name, phone, email (optional), password, confirm password, and the "I am 60 or over" checkbox. Each field shows its own error from the route's `errors` object.

> **THINGS TO KNOW**
>
> "Confirm password" has no `name`, so the server never receives it. It is checked only in the browser by `public/js/ui.js` (the "Passwords match" tick). Without JavaScript nothing compares the two boxes. This was deliberate, to avoid changing the server.

Used by	`src/routes/public.js` (GET and POST `/signup`)
Uses	partials `header`, `footer`, `langToggle`, `flash`, `phoneInput`, `ui/textInput`, `ui/passwordInput`; `public/images/illustrations/elders-chat.svg`

views/home.ejs

Home, in four sections: the **hero** (its own top bar, a big headline, the arch-cropped photo, the trust words, the number of upcoming events and the white "Find events" button), the **three choices** (Find events, Find seniors, Call), **Events near you** (the cards) and **How it works**.

Receives: `upcoming` (up to 4 events), plus the shared variables. The greeting uses `currentUser.firstName()` when someone is logged in.

Used by	`src/routes/public.js` (`GET /`)
Uses	partials `header` (with `pageLayout: 'hero'`), `footer`, `seniorNav`, `langToggle`, `eventTile`; `public/images/events/fitness.webp`; illustrations

`views/partials/eventTile.ejs`

The tall photo card in "Events near you". **Needs:** `event`. **Optional:** `variant` (`'dark'` or `'light'`), `eager` (load the picture straight away). It shows the category and "Free" pills, an info button that opens the date, place, category and organizer over the photo (a `<details>` element, so it works without JavaScript), the title with a tick, two lines of description, the date and time, and a **Register** button.

`views/partials/eventTile.ejs`, line 50

```
<class="tk-tag">a href="/events/<%= event.id %>" class="ev-join" aria-describedby="ev-<%= event.id %>-title"><%= t('stack.register') %><class="tk-tag">i class="fa-solid fa-plus" aria-hidden="true"></class="tk-tag">i></class="tk-tag">a>
```

→ Register opens the event's own page, where joining already handles every case (logged out, full, past, volunteering).

Used by	`views/home.ejs`
Uses	`views/partials/eventImage.ejs`; CSS classes `ev-shell`, `ev-card`, `ev-badge`, `ev-info`, `ev-body`, `ev-join`

The other shared partials

Partial	What it draws	Used by
`seniorNav.ejs`	The five-item menu; `placement: 'top'` or `'bottom'` (phones).	`header.ejs`, `home.ejs`
`nav.ejs`	The staff sidebar.	`header.ejs`
`langToggle.ejs`	A small form that posts to `/language` with the other language.	header, home, login, signup
`flash.ejs`	The one-time success/error/info box.	header and many pages
`eventImage.ejs`	Uploaded event photo, or the category cover (WebP with JPG fallback).	`eventTile`, `eventCard` and event pages

Partial	What it draws	Used by
`eventCard.ejs`, `eventTags.ejs`	The Find-events card; status tags ("Almost full").	`events/*`
`loginModal.ejs`	Log-in/sign-up pop-up when a visitor taps Register.	`events/index`, `social/find`
`confirmLogoutModal.ejs`	"Log out?" pop-up.	`footer.ejs`
`authAside.ejs`	Side panel for password pages.	`auth/forgot`, `reset`, `change`
`phoneInput.ejs`, `ui/textInput.ejs`, `ui/passwordInput.ejs`	Form boxes with icons; the password one has the eye button.	auth pages, `loginModal`, `admin/staff`
`qr.ejs`	The optional QR code, tucked away.	`events/confirmed`, `account/registrations`

The remaining pages in `views/events/`, `account/`, `social/` and `admin/` each belong to the route file of the same name; chapter 5 lists them all.

Browser scripts, styles, words and tests

File	What it does	Connected to
`public/js/ui.js`	Pop-ups (focus, Escape, backdrop), password eye, busy buttons, confirm-password check, log-out question. Never changes what a form sends.	loaded by `header.ejs`; reads `data-*` attributes in the views
`public/js/otp.js`	Six code boxes (typing moves on, paste fills them all) and a 60-second resend countdown.	`views/auth/verify.ejs`
`public/js/top-loader.js`	The thin loading bar.	`header.ejs` (`#top-loader`)
`public/js/phone-input.js`	Numbers only, 11 digits.	`header.ejs`, `phoneInput.ejs`
`public/js/swipe.js`, `dialog.js`	Swipe cards; pop-up accessibility.	`footer.ejs`
`public/js/chat.js`, `dashboard.js`, `upload.js`	Live chat; live dashboard numbers; upload button text.	`social/thread`, `admin/dashboard`, upload forms

File	What it does	Connected to
`src/styles/input.css`	The design system (chapter 9).	built into `public/css/main.css`
`locales/en.json`, `yo.json`	About 470 keys each: every sentence on the site, SMS and email text.	`src/services/i18n.js`
`src/sockets/index.js`	Live chat: checks the session and form token, only verified seniors, re-checks blocks on every message.	`server.js`; `public/js/chat.js`
`src/scripts/*.js`	Command-line helpers: `seed`, `db:check`, `create-admin`, `translate:ui`.	`package.json` scripts
`tests/*.test.js`	59 automated tests (chapter 5 says what each file covers).	`src/app.js`, `tests/helpers.js`

What you learned

- `server.js` sets up, `app.js` orders the request path, routes answer, services hold the rules, models talk to the database, views draw the page.
- The connection boxes tell you what else to check when you change a file.
- `capacity.js` is the heart of "never overbooked"; `locals.js` is why `t` and `currentUser` work in every template.

CHAPTER 8

How the main features work, end to end

Each feature as a numbered journey, naming the exact file at every step.

Creating an account

1. `GET /signup` → `src/routes/public.js` renders `views/auth/signup.ejs` with empty `errors` and `values`. If the visitor came from an event's Register button, a hidden `registerEventId` field remembers it.
2. In the browser, `public/js/phone-input.js` keeps the phone box to digits and `public/js/ui.js` checks that "Confirm password" matches.
3. `POST /signup` passes `signupLimiter` (5 an hour), then every field is checked:

   src/routes/public.js, line 110–114

   ```
   if (values.fullName.length < 2) errors.fullName = req.t('signup.error_name');
   if (!phone) errors.phone = req.t('phone.' + (phoneProblem(values.phone) || 'start'));
   if (values.email && !/^\S+@\S+\.\S+$/.test(values.email)) errors.email =
   req.t('signup.error_email');
   if (password.length < MIN_PASSWORD) errors.password = req.t('signup.error_password', {
   min: MIN_PASSWORD });
   if (!values.ageDeclared) errors.ageDeclared = req.t('signup.error_age');
   ```

   A phone number or email already in use is refused too.
4. With errors, the same page comes back (status 400) with one message under each wrong field and the typed values kept.
5. `User.create(...)` saves the account with a bcrypt password hash; `req.session.userId` logs them in; a "Welcome" flash message is set.
6. `finishAuth`: if they were registering for an event, confirm the phone first (`/verify`) or register straight away; otherwise go to the welcome video (`/onboarding`).

Logging in and out

1. `GET /login` → `views/auth/login.ejs`.
2. `POST /login` passes `loginLimiter` (10 tries per 15 minutes); `normalizePhone()` tidies the number and explains exactly what is wrong if it is invalid.
3. `User.findOne` and `bcrypt.compare`. On failure one general message is shown, never saying which part was wrong.

4. On success:

```
src/routes/public.js, line 175–181
```

```
if (user.role === 'admin') {
  return res.redirect(returnTo && returnTo.startsWith('/admin') ? returnTo :
'/admin');
}
const after = afterAuth(req.body);
if (after.registerEventId || after.next) return finishAuth(req, res, user, after,
'/');
if (!user.onboardingSeenAt) return res.redirect('/onboarding'); // e.g. a relative
signed them up
res.redirect(returnTo || '/');
```

   Staff go to `/admin`; a senior finishes any event registration, sees the welcome video if never seen, or returns to the page they wanted.

5. `src/middleware/locals.js` loads the user on every later request; the cookie lasts 30 days.
6. Log out: `public/js/ui.js` asks "Log out?" (`partials/confirmLogoutModal.ejs`), then `POST /logout` ends the session and goes to `/login`.

Confirming the phone number

1. Joining an event or messaging needs a confirmed phone. `needsContactCheck()` in `src/routes/verify.js` remembers what they were doing and sends them to `/verify`.
2. `POST /verify/send` → `src/services/verification.js` sends a 6-digit code by SMS (in dry-run mode it is printed in the terminal). `public/js/otp.js` shows six boxes and a 60-second resend countdown.
3. `POST /verify/check`; once confirmed, `carryOn()` finishes the registration they started.

The home page, hero and Events near you

1. `GET /` → `src/routes/public.js` loads up to 4 upcoming published events with their organization, translates them if needed, and renders `home`.
2. `views/home.ejs` includes the header with `pageLayout: 'hero'`, draws the hero (own top bar, headline, photo `public/images/events/fitness.webp`, event count, "Find events" button).
3. It loops over `upcoming` and includes `partials/eventTile.ejs` per event, alternating dark and light; with none, an empty-state message.
4. Each card uses `partials/eventImage.ejs` (uploaded photo or category cover), a `<details>` info panel, and a Register link to `/events/:id`.

Finding, filtering and joining events

1. `GET /events?category=...` → `src/routes/events.js` loads open events, removes full ones and ones already joined:

   src/routes/events.js, line 63–66

   ```
   const open = events.filter((e) => {
     const taken = counts[e.id] ? counts[e.id].attendee : 0;
     return taken < e.capacity && !mine.includes(e.id);
   });
   ```

   then skips those in `req.session.skippedEvents` and shows one card (`views/events/index.ejs`).

2. A visitor pressing Register gets the log-in pop-up (`partials/loginModal.ejs`); after logging in they are registered automatically.

3. A senior posts to `/events/:id/register`; `registerAndNotify()` calls `capacity.register()`, which locks the event and checks it is open and not full:

   src/services/capacity.js, line 72–75

   ```
   const taken = await countTaken(eventId, type, transaction);
   if (taken >= capacityFor(event, type)) {
     return { ok: false, reason: 'full' };
   }
   ```

4. A Registration with a ticket number is saved, the SMS/email confirmation is sent, and the "You're registered" pop-up opens.

5. Cancel from `/registrations` (`src/routes/account.js`); reminders go out at 8:00 (`src/services/cron.js`); staff mark attendance through `src/services/attendance.js`.

What you learned

- Sign up and log in live in `src/routes/public.js`; the session remembers `userId`.
- Home's cards come from one query and `eventTile.ejs`; Register leads to the event page.
- Every registration goes through `capacity.register()`, which cannot overbook.